Language Representation Learning and Text Classification: A Comparative Study and Cost-Benefit Analysis

by Peter J Worth & Dr. Ionut Cardei
Florida Atlantic University
Dept of Electrical Engineering and Computer Science

Spring 2025

1 Table of Contents

1 TABLE OF CONTENTS 1-2

2 TABLE OF FIGURES 2-3

3 ABSTRACT 3-4

4 PREAMBLE & RESEARCH CONTEXT 4-5

4.1 Unique Contributions of this Work 4-6
4.2 Research Context: Text Classification in ML and DL 4-8
4.3 Structure of this Paper 4-9

5 EXPERIMENTAL DESIGN: TEST SCOPE & DATA FLOW 5-11

5.1 Supported Language Models 5-12
5.1.1 Static Embedding Models 5-13
5.1.2 Context-Aware Language Models (Transformers or LLMs) 5-14
5.2 Supported Datasets 5-17
5.2.1 Reuters21578: multi-label, 115 classes 5-18
5.2.2 BBC News: single-label, 5 classes 5-18
5.2.3 RCV1-V2: multi-label, 103 classes 5-18
5.2.4 20 Newsgroups: single-label, 20 classes 5-18
5.2.5 arXiv: multi-label, 58 classes 5-19
5.2.6 arXiv protoformer: single-label, 10 classes 5-19
5.2.7 OHSUMED: multi-label, 23 classes 5-19
5.2.8 IMDB: single-label, 2 classes 5-19
5.2.9 Dataset Summary Table 5-19
5.3 Supported Metrics 5-20
5.4 Language Representation and Test Workflow 5-21
5.4.1 Machine Learning Approaches (Baseline Testing) 5-21
5.4.2 Neural Models (Deep Learning) 5-25

6 TEST DATA DIMENSIONAL ANALYSIS 6-33

6.1 Baseline ML Results 6-33
6.1.1 By Classifier 6-33
6.1.2 By Dataset 6-35
6.1.3 By Representation Form 6-36
6.1.4 By Language Model 6-38
6.2 Neural Models (Deep Learning) 6-41
6.2.1 By Language Model 6-41
6.2.2 By Classifier 6-43
6.2.3 By Dataset 6-44
6.3 Global Analysis 6-50

7 SUMMARY & CONCLUDING REMARKS 7-56

7.1 Areas of Promising Research 7-57

8 BIBLIOGRAPHY 8-59

2 Table of Figures

FIGURE 1: STATIC NEURAL MODEL ARCHITECTURE (LSTM) 5-12
FIGURE 2: LANGUAGE MODEL SUMMARY TABLE 5-17
FIGURE 3: STATIC LANGUAGE MODEL CLASSIFIER (CNN ARCHITECTURE) 5-27
FIGURE 4: LCCNNBERTCLASSIFIER ARCHITECTURE (TRANSFORMERS) 5-30
FIGURE 5: MACHINE LEARNING CLASSIFIER PERFORMANCE SUMMARY 6-34
FIGURE 6: ML CLASSIFIER SUMMARY: BY REPRESENTATION 6-37
FIGURE 7: LANGUAGE MODEL PERFORMANCE SUMMARY - ML CLASSIFIERS 6-39
FIGURE 8: NEURAL CLASSIFIER LANGUAGE MODEL TYPE PERFORMANCE SUMMARY 6-42
FIGURE 9: NEURAL MODEL CLASSIFIER PERFORMANCE SUMMARY 6-43
FIGURE 10: MACRO-F1 SCORES FOR NEURAL CLASSIFIERS (OHSUMED DATASET) 6-45
FIGURE 11: MICRO-F1 SCORES FOR NEURAL CLASSIFIERS (OHSUMED DATASET) 6-46
FIGURE 12: MACRO-F1 VALUES ACROSS NEURAL CLASSIFIERS (RCV1 DATASET) 6-47
FIGURE 13: MICRO-F1 VALUES ACROSS NEURAL CLASSIFIERS (RCV1 DATASET) 6-48
FIGURE 14: ARXIX NEURAL CLASSIFIER PERFORMANCE COMPARISON (MACRO-F1) 6-49
FIGURE 15: ARXIX NEURAL CLASSIFIER PERFORMANCE COMPARISON (MICRO-F1) 6-50
FIGURE 16: GLOBAL CLASSIFIER PERFORMANCE SUMMARY 6-51
FIGURE 17: GLOBAL PERFORMANCE SUMMARY BY LANGUAGE MODEL 6-52

3 Abstract

The introduction and widespread adoption of LLMs over the last few years has changed the entire technological landscape, ushering in a wave of transformation (literally and figuratively) that rivals the changes brought about by the advent of the Internet in tandem with broadband networking at the turn of the century. It's not at all clear if and when the pace of the evolution of this technology, driven by billions of dollars in investment on both the software and hardware front by some of the largest technology companies in the world, will abate and in the interim, there is tremendous enthusiasm and speculation about what this technology is capable of. What we look to do with this work, affectionately called Layer Cake after the name of the software platform we developed to support this research aims to bring some of this other-worldly enthusiasm related to LLMs down to earth somewhat, by analyzing a wide array of language models within the context of *text classification*, one of the most fundamental and well-studied problems in NLP.

In this research, we use Layer Cake to test both static pretrained language models and a few different variants of LLMs, which are in effect transformer architected language models, with a range of datasets, classifiers, and hyperparameter settings, with a focus on the different ways the underlying (sequential) text is represented in the various test scenarios, a study that fits just as much into the field of *language representation learning* as much as it does text classification and NLP in fact. Given the breadth and depth of the testing we perform with Layer Cake, we shed light on not just the comparative performance of different classifiers across the ML and DL landscape, but also on the underlying computational complexity, and inferred cost, of working with LLMs in NLP, a topic of profound implications for anyone working with this technology from a research and/or development perspective. Our findings highlight that while the transformer models are in fact the best performers in almost all of our test scenarios, nonetheless in all but one of the test cases there exists a classical ML, or static neural model approach which performs almost as well as the LLM, each of which come with significantly less computational costs, a factor which scales geometrically with language model size and complexity given the dreaded curse of (computational) dimensionality.

Keywords:

Language Representation Learning, Text Classification, NLP, Word Embeddings, Vector Space Models, LLMs, Language Models, Semantic Spaces, Deep Learning, Machine Learning, Artificial Intelligence

4 Preamble & Research Context

Certainly with respect to how Alan Turing formulated the problem in 1950 (Turing, 1950), the current state of AI is not just indistinguishable from a moderately intelligent human being, the current state of AI rivals in many respects some of the most highly intelligent human beings on the planet – at least from an informational breadth and basic analytical perspective. There nonetheless remain aspects of human intelligence that still remain elusive for AI, and hence the use of the moniker *artificial general intelligence*, or simply AGI, which serves as the north star for almost all of the major technology companies investing in AI right now.

The form within which this intelligence takes is perhaps quite different than Turing envisioned, but given how the technology has been packaged and encapsulated - i.e. *large language models*, or simply LLMs, which are in effect highly sophisticated token embedding machines operating in an extraordinarily high (feature) dimension space – we can certainly expect over the next few years the rapid advance of the integration of these more advanced models into more and more advanced mechanical devices, driving another massive technological wave in robotics that we are just beginning to witness. Many of these machines will, and to a certain extent already do, follow a humanoid design which will ultimately leave us with the challenge that has been the subject of countless science fiction novels for the past century, where humanoid looking robots for all intents and purposes possess a form of intelligence that is almost indistinguishable from human intelligence in most contexts.

The technological advancements necessary to facilitate these developments in AI however are not limited just to LLMs, which are a very advanced form of dense *language representation* effectively. These advancements in the breadth, scope and power of LLMs are possible only due to massive investments and technological advancements in the digitization of the global economy, the underlying network and system infrastructure that supports such a digitized global economy, distributed computing technology that allows for the training and construction of these massive models against a vast array of data across hundreds and sometimes thousands of micro-processors, advancements in the speed and memory capacity of the underlying micro-processors themselves (i.e. GPUs), and many other groundbreaking technologies which facilitate the research and development of this revolutionary technology – a technology that in and of itself is driving the pace of innovation given what it is capable of.

All of these technological advancements have led to radical changes not only with respect to how humankind conducts and transacts business throughout the globe, but also – among other things - with respect to how we interact with each other, and even how nation states and corporations govern their constituents. It is fair to say that there are very few aspects of human life that have not, or will not, be impacted by the advent of AI in the coming years if they have not been radically changed already. That being said, the research field that sits at the very front edge of this wave of transformation is with respect to large scale transformer models that leverage deep learning to build lever more sophisticated representations of natural language in more and more complex high dimensional geometrical configurations, i.e. what we call *semantic space* following nomenclature from linguistics and cognitive science (Worth, 2023).

Advancements in this field rest on many different technologies which underpin their capabilities but three in particular stand out as fundamental and necessary conditions for the explosion in capabilities we have witnessed from these models the last few years:

i. deep learning (DL) techniques that drive various forms of sophisticated language representation,

ii. transformer, attention-based DL models that drive sophisticated forms of language representation in high dimensional space,

iii. advancements in computational power and parallelism that facilitate computation of massive models over massively distributed computing environments

It is these three areas of R&D which have underpinned this AI revolution we are witnessing more than anything else which provide the computational architecture and infrastructure that transforms computers into devices that exhibit no less than revolutionary capabilities that are not just indistinguishable from human intelligence but arguably have begun the process of redefining the limits intelligence writ large.[1]

This work focuses on the underlying data structures and techniques which generate the language representation forms that have proven to be so effective when used with these LLMs, as well as requisite computational resource requirements for fine tuning these LLMs for a very specific task – namely classification, one of most fundamental NLP problems that crosses into many facets of research and commercial industry which has been a staple of NLP for decades now.

4.1 Unique Contributions of this Work

While there is much published research related to state of art results in NLP associated with a veritable potpourri of LLM architectures, with this work we introduce a much broader analytical and comparative perspective to the problem, focusing on different classifiers from both the ML and DL research domains and therefore different computing architectures, tested in with a broad range of language model types, each of which are constructed with their own specific training methodology and tokenization strategies. That broader perspective allows for a much more comprehensive and holistic analysis of the different approaches to text classification through which the performance of LLMs themselves can be viewed and by which the most important levers in the entire classification problem domain can be viewed.

Furthermore, with this research, we have tested wide range of language representation forms within the context of our ML model testing framework, another area of model design that is oft overlooked in the text classification and NLP research literature, especially with LLMs given that they, along with almost all neural models that are used in NLP, presume a fixed language representation form as the basis of their models.[2] Almost all the research around LLMs that most of the research in this area, particularly with respect to the neural transformer models, presumes that the language model representation is either a) a given, or b) not relevant from a performance standpoint given that it is typically the same across all different neural

[1] Ironically the best method for distinguishing between human intelligence and artificial intelligence at this point is AI itself.

[2] With LLMs, and in fact with almost all neural models in NLP, the underling (natural) language is converted into a fixed size (max_width) sequence of token indices where each token is represented by a vector of real numbers of a size fixed by the language model (aka hidden_size) that is the result of the pretraining of that language model against a fixed pretraining dataset – i.e. what are known as *embeddings* but which are extensions of Vector Space Models which have been around since the 1970s. Each LLM creates these embeddings by defining the underling vector space dimensions (as a form of feature reduction from the total dictionary size of the pretrained corpus) and then learning the embeddings for each token using the language model type and variant specific (neural) architecture and algorithm (loss function, etc.).

architectures / transformer language models. This assumption is inferred from the lack of coverage this aspect of NLP when it comes to deep learning at least.[3]

Given the focus on LLM performance across the problem landscape in most research in NLP the last few years, this recentering of the analysis around different language representation forms as both inputs to the classifier models (embedding layers) and as outputs of different DL architectures (i.e. embeddings), allows for an analysis of the results from the perspective of an optimization problem in high dimensional semantic space, a semantic space that is defined by the dataset representation form and the language model representation form – a combination of the two really – which is precisely what is going on mathematically within the context of the text classification use case that is the subject of this work. Some of these results therefore, due to their abstract mathematical and geometrical foundations, should be applicable to other NLP problems that leverage text vectorization, embedding or transformer-based language representation modes – which essentially describes the entire NLP landscape.

This work looks at the problem of text classification not just with respect to the different types of text classifier models – i.e. ML versus DL approaches – but also looks at, and experiments broadly with, various forms of language representation above and beyond just the different language models themselves (each of which actually does represent a different form of language representation), particularly in the case of our ML model testing framework. For example, we find is that both text vectorization used in tandem with LSA for feature reduction in conjunction with WCEs, i.e., Word-Class Embeddings, as well as pretrained embedding representation forms (another form of feature reduction) in conjunction with WCEs, when used as inputs to classical ML classifiers like Support Vector Machines (SVM) and Logistic Regression classifiers, both provide very competitive results relative to the more complex and computational intensive neural classifiers that we include in Layer Cake – both in the static language model variants that we test with, as well as the dynamic, transformer based language models we test with. Given the focus of the research community on LLM performance in the last few years, these results should give practitioners pause before selecting LLMs as the de facto standard for any and all NLP problems.

Furthermore, in the testing of different neural classifier variants to support both static language models as well as transformer, context aware models, we discovered a significant difference in the flexibility of the different neural architectural models, with the transformer embedding layer being tightly coupled with its computational logic (as one might expect perhaps), which significantly limits the ability to optimize the model – like for example using feature, or dictionary size, reduction techniques as we do for the other classifiers. This constraint, limits how and where in model training one can add pretrained embedding structures that are extra-model or are trained outside the transformer language model context that we are using for the task at hand. In our testing for example, we could not find an approach where the integration of Token-Class Embeddings, or TCEs (the transformer variant of WCEs), actually increased the effectiveness of the model, in fact we found the opposite, that adding these TCEs with the context-aware embeddings during training - no matter how we added them or how we normalized the underlying embedding dimensions - actually made the model less effective.

Also, in addition to this unique perspective and unique set of findings, we also produce within the context of this work a rich set of results that covering a broad spectrum of metrics[4] that is available for further

[3] The ML literature is filled with feature reduction techniques and approaches and optimizations, speaking to the relevance and importance that various forms of language representation have on model training times and overall model performance efficacy.
[4] We report macro-f1, micro-f1, precision, recall, accuracy, Jacard-Index and Hamming loss metrics for each test run.

analysis that is generated using a cross-platform, extensible, benchmarking platform that supports a variety of language models, classifiers and datasets that can be extended for further insights into this field of research, i.e. text classification, or even extended to support other NLP problems like sentiment classification, topic modeling or even Natural Language Understanding (NLU) benchmarking if need be. This is Layer Cake in a nutshell.[5]

Given the plethora of model variations we test within the context this research, and the breadth of data we capture related to model training and performance, Layer Cake gives us a unique perspective into not just how the language models themselves compare with each other across different classifiers, hyperparameter configurations, different language models and types across different datasets, but also reveals clear metrics around model training times so that we have an opportunity to look at model effectiveness *in conjunction with model system dependencies and computational complexity*, putting us in a position to evaluate the *cost-benefit* of LLM model performance improvements that are so widely cited in the literature and touted in the market at present.

4.2 Research Context: Text Classification in ML and DL

Text classification, the core NL problem that is analyzed and tested in this research is one of the foundational problems in Natural Language Processing (NLP) that has applications across the commercial and application landscape ranging from topic classification for news and other related research sites and frameworks, to spam detection and filtering, to content moderation and compliance (sentiment analysis) to fraud prevention and malware detection in the cyber security space. The problem, as we frame it here, involves assigning predefined labels to textual data and then building and training models to predict how new text (documents) are to be classified based upon historical data that has been fed into the model, i.e. a form of *supervised training* which is a very common, and very effective, means of model training and construction.

The text classification research landscape today, like most problems in NLP in fact, is heavily influenced by LLMs, which many of the tech giants have made massive investments in. This latter point, while painstakingly obvious for many who at some level pay attention to the news, is remarkably left out of the research literature implying that is satisfactory to view model effectiveness gains in a vacuum irrespective of the time, or cost, necessary to achieve these results.[6] It's worth mentioning that the Quantum computing research community suffers from the same delusion. To be clear, we are not suggesting that there isn't extraordinary value to the R&D of the technology in question just simply leaving out the cost-benefit analysis piece of the equation is at best intellectually remissive, but a more accurate description of the oversight would be that it *substantively misleading*. This work intends to address these shortcomings by shedding light on the very metrics that have been mostly left out of the research literature with respect to the underlying cost associated said best-in-class results using technology that almost all of the major global tech giants have made enormous bets on.

---

[5] Layer Cake supports Mac MPS GPU architectures as well as Ubuntu Linux GPU architectures with the exception being the DeepSeek models which are not supported (yet) on MacOS. Source available on GitHub at Source code for Layer Cake available at https://github.com/pworth1971/layer-cake

[6] According to many reports, as of the end of 2024 the daily run costs of ChatGPT were in the range of $700,000 per day and were mostly related to a) state of the art GPUs required to train and tune the model(s), and b) storage and servers hosted on Azure. See for example https://www.digitaltrends.com/computing/chatgpt-cost-to-operate/.

This study, one of the most comprehensive and largest of its kind both in terms of the breadth and scope of the underlying types of classifiers that are tested, as well as the different types of language models that are tested in addition to the dimensional analysis and reporting that we generate as a byproduct of this research is nothing less groundbreaking with respect to paving the way for an open dialogue on a dimension of analysis of this technology that while not totally absent from the research literature is nonetheless extremely underserved given the oversized level of importance these factors have in achieving the state of the art performance which most of these studies point to. If there ever existed an elephant in the room type of situation in an area of prominent computer science research, this would be it.

With this study, what we call *Layer Cake* using the name of the platform that was developed to generate the test data that we analyze within the context of this work, we specifically call attention to real data and metrics that look at:

i. how much performance gain can one expect by using LLMs (i.e. contextually aware, dynamically computed token embedding generators), over classical machine learning approaches to text classification (such as SVM and Logistic Regression models),

ii. what kind of performance gain do the transformer embedding models have over their static counterparts such as GloVe, Word2Vec or fastText, and c) what is the computational cost for said gains.

iii. what is the underlying computational cost of said gains in terms of model training and construction times and dependent systems infrastructure and hardware (dominated by GPU costs), and

iv. what forms of language representation are most performant and effective within the context of the different types of models in scope for our work.

There have been a variety of good research surveys written that shed light on this topic, albeit somewhat dated given the rapid advancements in the field, with particular focus on Pretrained Language Models for NLP broadly published in 2020 (Qiu, et al., 2020), Deep Learning models with respect to Text Classification specifically (Minaee, et al., 2021), and perhaps most applicable with respect to the work here a survey for traditional ML and Deep Learning methods for text classification published in 2022 (Li, et al., 2022).

4.3 Structure of this Paper

The rest of this paper follows the following basic outline:

- description of test setup and scope, followed by
- detailed description of the test workflow, forms of language representation and classifiers used in testing, followed by
- detailed analysis of the data which looks at the ML and DL approached both independently and then together globally, and then
- a summary analysis which includes recommended areas of further research, lastly
- a citation list (Bibliography).

The analysis parts of this paper are dominated by charts and graphs that visualize the results according to the key dimensions that represent the major themes of this work – namely classifier architecture, language representation form, and training time and computational complexity.

5 Experimental Design: Test Scope & Data Flow

Our experiments consist of three separate sets of classifiers that are run with various hyperparameter settings and configurations, each tested with different language models and datasets, with all metrics and analytics written to a log database for later analysis. Our test methodology follows (Moreo, et al., 2021), but we extend the footprint significantly in terms of the ML classifiers and language models we test, extending the analysis to transformers.

Layer Cake consists of four different modules – three for classifier testing and another for reporting & analytics. While the system is designed to support CPU, GPU and MPS (Apple MacOS M chips) based architectures, the final set of test runs were performed on a Linux Ubuntu system[7] with the following specs:

- OS: Ubuntu Linux version 22.04, x86 architecture
- CPUs: 64 physical, 128 logical; 512 GB RAM (memory)
- GPUs: 4 NVIDIA RTX 6000 Ada Generation, 48GB of Memory each
- CUDA 12.4
- Python 3.10 (DeepSeek model dependencies)
    - PyTorch 1.4.1
    - Transformers 1.45.2
    - Sklearn 1.6.1

The ML baseline testing module (ml_classification_test_version.py) module covers the baseline machine learning classifiers that we test with, namely Support Vector Machines and Logistic Regression and Naïve Bayes classifiers. We use the *sklearn* APIs to build and run the respective model, namely the *LinearSVC*, *LogisticRegression*, and *MultinomialNB*, using a *OneVsRestClassifier* approach for the multi-label datasets. This code is CPU bound for the most part, although the transformer embedding computations of the different dataset and language model combinations are GPU bound. This code ran on a single GPU system with the same spaces as described above.

The neural model classifiers that are designed for the static embeddings are GPU bound (CUDA), although they also run on MPS / Apple Silicon. These do not support parallelism and are designed to run on a single GPU. The transformer models however (*trans_layer_cake_version.py*), which leverage the Hugging Face libraries, do support a simple form of parallelism for PyTorch known as Data Parallelism, where the model is replicated on all of the in-scope GPUs, data is doled out equally across the GPUs during training, in parallel, and then model updates are synchronized after each training step. While limited in terms of the size of model that can be supported (we had to use smaller models for Layer Cake due to this limitation), it is nonetheless easy to set up certainly faster than using a single GPU for training. [8]

One of the insights that falls out of this analysis is that the nature of language representation form drives the model architecture, and vice versa. For example, when using the transformers, the first set of layers in the model are the underlying transformer model itself which facilitates the computation of the embeddings, rather than a static, fixed embedding layer which is used with the word based, static embedding models

[7] RCV-V1 testing had to be done on MacOS, another Unix variant, given the memory requirements. DeepSeek model numbers for RCV1 therefore were not computed given that the DeepSeek models can only be worked with on Linux environments at the time of this writing.

[8] For a more detailed treatment of parallelism using PyTorch and/or Hugging Face we suggest https://huggingface.co/docs/transformers/v4.13.0/en/parallelism.

(Word2Vec, FastText, GloVe). This has significant performance implications, a fact that is well known – what is less well known is clear quantification of the cost-benefit analysis of utilizing these more complex and computationally heavy approaches to text classification. Quantifying this, which we look to do here with this work to a large extent, has implications well beyond just problems of text classification.

Having said that, our DL classifiers consist of two distinct categories of models that are designed for two distinct type of *embeddings* – the first designed for static pretrained embeddings that uses very thin PyTorch crafted CNN, ATTN and LSTM (from (Moreo, et al., 2021), and then a separate module that that is designed specifically to test the HuggingFace transformer classifiers. This code is GPU bound but it does not support parallelism, i.e. it runs on a single GPU.

```
NeuralClassifier(
  (embed): EmbeddingCustom(
    (pretrained_embeddings): Embedding(22846, 300)
  )
  (projection): LSTMprojection(
    (lstm): LSTM(300, 128)
  )
  (label): Linear(in_features=128, out_features=5, bias=True)
)
```

Figure 1: Static Neural Model Architecture (LSTM)

The dynamic, transformer based pretrained language model testing on the other hand, which is designed to test all of the HF transformer language models – BERT, RoBERTa, DistilBERT, XLNet, GPT-2, Llama, DeepSeek - does in fact support (the most basic form of PyTorch) parallelism.[9]

What is important to understand with respect to the underlying computational requirements of the different LC modules, especially within the context of the model construction time analysis below, is that:

i. the ML testing is CPU bound,

ii. the neural model testing is all GPU bound, single GPU, and

iii. the transformer model testing supports GPU parallelism (4 GPUs)

In other words, our transformer models used a 4 GPU environment whereas our ML baseline script, and our static pretrained neural model classifiers, only depend on a single GPU, an important distinction we will call out in the analysis section below when we look at model training and testing times.

5.1 Supported Language Models

[9] We made several attempts at getting Word-Class Embeddings, really their tokenized counterparts, integrated with various HF Classifiers, these efforts were met with results that indicated quite clearly – regardless of the method of integration – that at least class level embeddings could not be effectively integrated into the transformer architected model.

Layer Cake supports a wide array of language models variety of models which gives us an opportunity to compare different forms of language representation in different contexts and look at how, and what, a language model was trained on, and to what extent those factors bear on classifier effectiveness.

5.1.1 Static Embedding Models

For the static language models that we tested, whose embeddings are word based as opposed to token based as the transformer models are, we used a GloVe model, a Word2Vec model and a FastText model, each of which was trained on the Common Crawl dataset which consists of a massive, open source web archive that consists of petabytes of data collected by crawling the web.[10]

5.1.1.1 GloVe: glove.840B.300d.txt

glove.840B.300d.txt is the GloVe pretrained word embeddings set that was trained based on global co-occurrence statistics against the common (web) crawl dataset which includes 840B tokens and 2.2 million vocabulary words, is of dimension (vector size) 300, and around 2GB all in. The GloVe specific training method focuses on co-occurrence statistics from a word-context matrix across the entire corpus, capturing both syntactic and semantic relationships and was introduced in 2014 (Pennington, et al., 2014).[11]

5.1.1.2 Word2Vec: GoogleNews-vectors-negative300.bin

GoogleNews-vectors-negative300.bin is the Word2Vec trained static word embeddings, originally trained by Google, which is also 300 dimension vectors, contains over 3 million words and phrases and is all in about 1.5 GB. The embeddings were trained from the Google News dataset, hence the name, and was built using the Word2Vec model, introduced in 2013 by (Mikolov, et al., 2013). Specifically, the model is trained using the Continuous Bag of Words (CBOW) and skip-gram methods, where the skip-gram model predicts surrounding context words given a center word, and negative sampling is used to optimize training by selecting a few "noise" words during prediction instead of updating the entire vocabulary.

5.1.1.3 FastText: crawl-300d-2M.vec.bin

crawl-300d-2M.vec.bin is the binary format of the FastText static embedding variant we test with that is also of size 300 (dimensions of embedding vector), contains around 2M unique words, about 6GB all in and was also trained on the Common Crawl dataset, preprocessed to remove noisy content, duplicates, and non-linguistic data (e.g., scripts, images, ads). This fastText language model was trained using a skip-gram

[10] The Common Crawl dataset, as used by FastText crawl-300d-2M trained language model which is used in this study contains 600+ billion tokens, making it one of the largest public corpora available for training NLP models. The dataset is multilingual, with content spanning hundreds of languages and while English is dominant, significant amounts of content are curated for other major languages like Spanish, French, and Chinese. The data is sourced from a wide range of websites, covering topics such as news articles, blogs, social media, scientific articles, forums and community sites and technical documentation, representation a broad range of data that supports the training of language models that cover the broad scope of human digital) language. The main goal of the Common Crawl project is to provide an open-access, large-scale web dataset for academic, industrial, and research use. Common Crawl data is publicly available and constantly updated through periodic crawls. Source: https://commoncrawl.org/

[11] https://nlp.stanford.edu/projects/glove/

method with negative sampling, trained on word context, and incorporating subword modeling (n-grams). FastText splits each word into character n-grams and builds word representations by summing subword vectors which enables it to handle OOV words by generating approximate embeddings based on their subword components. The model we use lacks the subword information but is trained to that effect. (Bojanowski, et al., 2017)

5.1.2 Context-Aware Language Models (Transformers or LLMs)

For the transformer tests, we use three different BERT variants, XLNet and GPT2, as well as a small LLaMA model and a small, distilled DeepSeek model, each of these models is briefly described below.

5.1.2.1 BERT, RoBERTa and DistilBERT

BERT and RoBERTa are pretrained on massive amounts of text data using masked language modeling (MLM) and (for BERT) next sentence prediction (NSP). During pretraining, part of the sentence is hidden (masked), and the model is trained to predict the missing word. MLM introduces probabilistic predictions into the model's learning process. The embeddings that result from MLM are optimized for downstream tasks, such as classification, named entity recognition, or question answering.

BERT considers the entire context of a word from both directions (left-to-right and right-to-left) using the self-attention mechanism. This means that the embedding for a word in BERT is influenced by both the preceding and succeeding words in a sentence, making the embeddings contextual and *bidirectional*. RoBERTa builds on BERT but removes the next sentence prediction (NSP) task and introduces changes to pretraining hyperparameters, such as training with larger batch sizes and more data. This leads to better generalization performance.

RoBERTa, on the other hand, uses dynamic masking during pretraining, meaning the mask positions are selected randomly each time a sequence is fed into the model. This leads to more robust embeddings. It also uses *dynamic masking*, where the masked tokens in a sequence change across training epochs, making the training more robust and, like its predecessor BERT (and distinct from traditional RNNs that look at words sequentially or autoregressive models like GPT which internally mask future tokens), allows the model to construct a bidirectional, contextually aware, representation of a given sequence of tokens which creates better embeddings, i.e. a richer language representation form ultimately as illustrated on its performance on a wide range of NLU and NLP tasks and benchmarks.

In the embedding space, BERT and RoBERTa learn to position words in a way that encodes both syntactic and semantic relationships, but these relationships are deeply tied to the specific context. In BERT/RoBERTa's embedding space, the distance between words is not just based on their isolated meaning but on how they interact with other words in a given sentence or passage. Thus, the same word can have multiple geometric positions depending on the sentence in which it appears. BERT and RoBERTa embeddings are contextual, probabilistic, and dynamic.

DistilBERT was pretrained on the same corpus as BERT but, using a technique called *knowledge distillation*, which is when a model uses a source model as a target (aka *teacher model*) and learns to mimic its outputs while not requiring the full training computational resources, yielding a smaller model that is comparable in performance. More specifically from a distillation perspective, DistilBERT uses a masking and cosine

embedding loss algorithm to generate hidden states as close as possible as the BERT base model while again not requiring the full ensemble of training and parameters as its teacher model, yielding a model that is around 40% smaller than BERT and yet still retains some 97% of its performance across a variety of NLP and NLU benchmarks making it an ideal model to use on edge network devices who do not have access to a lot of computing power for example.

5.1.2.2 XLNet

XLNet is another transformer-based model that we use in Layer Cake that incorporates aspects of both encoder and decoder architectures but functions more like a generalized autoregressive model designed for bidirectional context understanding (Yang, et al., 2019). XLNet incorporates ideas from Transformer-XL (Dai, et al., 2019), the state-of-the-art autoregressive model, into the pretraining objective of BERT by maximizing the expected likelihood over all permutations of the input sequence tokens.

XLNet takes a hybrid approach to training, leveraging advantages of both BERT (bidirectional encoding) and GPT (autoregressive generation). XLNet is neither strictly encoder-only nor decoder-only, instead, it is built on the Transformer-XL framework, which uses a permutation-based autoregressive training objective that enables it to model bidirectional context without the need for masking (as in BERT). While it primarily focuses on understanding text (like encoder-only models), its architecture and training objectives are inspired by decoder-based autoregressive models.

XLNet predicts tokens based on previous tokens but does so in a permuted order. This allows the model to capture bidirectional context in a flexible way without relying on masking like BERT. Unlike BERT's Masked Language Modeling (MLM), XLNet models all positions of the sequence in random permutations during training, making it a generalized autoregressive model. XLNet extends Transformer-XL, which introduces a memory mechanism to learn dependencies over long sequences. This allows XLNet to handle longer contexts than standard Transformers like BERT.

5.1.2.3 GPT-2 (Generative Pretrained Transformer)

GPT-2 (Generative Pretrained Transformer 2) is an unsupervised, *auto-regressive*, language model from OpenAI introduced in 2020 in the paper "*Language Models are Few-Shot Learners.*" (Brown, et al., 2020). GPT2 is an English language model also uses the Transformer architecture, decoder only, leveraging a *causal*, or *auto-regressive* approach to training where tokens are predicted sequentially and are conditioned on previously conditioned tokens.

More precisely, inputs are sequences of continuous text of a certain length and the targets are the same sequence, shifted one token (word or piece of word) to the right. The model uses internally a mask-mechanism to make sure the predictions for the token *i* only uses the inputs from 1 to *i* but not the future tokens, allowing the model to construct an inner representation of the English language that can then be used to extract features useful for downstream tasks.[12]

[12] https://huggingface.co/openai-community/gpt2

GPT-2 is designed to generate coherent, human-like text when given a prompt, i.e. *language generation*, but it can be fine-tuned for a wide range of downstream NLP tasks such as text completion, summarization, translation, and question answering and provides a good contrast model to the bidirectional, masked BERT models we test with, as well as a comparison to the other language generation models that we use in in Layer Cake (Llama and DeepSeek, more on these models below).

5.1.2.4 LLaMA (Large Language Model Meta AI)

Llama, or *Large Language Model Meta AI*, is the name for a family of open-source language models created by Meta (formerly Facebook), that was introduced in 2023 in the paper "*LLaMA: Open and Efficient Foundation Language Models*" (Touvron, et al., 2023). These are multi-lingual models whose tuned versions use supervised fine-tuning techniques (SFT) and reinforcement learning with human feedback (RLHF) to align utility goals with safety and security objectives.[13]

LLaMA models are - similar to GPT2 - transformer based, decoder only, and trained auto-regressively. Like GPT2 they are also designed for language generation tasks primarily, i.e. what the market has come to understand and call LLMs. Its transformer-based architecture with decoder layers is optimized for scaling and it uses sparse attention mechanisms to handle longer documents effectively, focusing on dense token prediction during its auto-regressive token generation scheme. Given its underlying architecture and training design, the model is best suited for General-purpose tasks, including long-form generation, summarization, and document retrieval.

5.1.2.5 DeepSeek

DeepSeek is a Chinese company that has recently released a few families of language models that have made a significant impact on the overall LLM landscape given the efficiency gains in training that they have gained along with the effectiveness of their models according to a variety of NLU and NLP benchmarks. DeepSeek models use large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step and have demonstrated remarkable performance on reasoning that are comparable to LLaMA and OpenAI models.[14]

These DeepSeek models are transformer based with both encoder and decoder aspects, and the R1 family in particular is designed for *reasoning* capabilities, a relatively new capability that is being actively worked on in the LLM community which allows the model to break tasks into components and solve problems sequentially and with dependencies, what is known as chain-of-thought (COT) capability in the research community.

DeepSeek has also produced several LLaMA and Qwen[15] based distilled models, the latter of which (smallest version) we incorporate into this this study, which remain effective for a variety of NLP and NLU tasks (as we see here) and are computationally efficient as well given the distillation-based design.

[13] https://huggingface.co/meta-llama/Llama-3.2-1B
[14] https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B
[15] Qwen is an open-source family of language models developed by Alibaba Cloud in 2023 in the paper (Bai, 2023)

5.1.2.6 Summary Table

The specific language models we use for Layer Cake testing along with details on their underlying architecture, what they are (pre) trained on, and other salient information, are summarized in the figure / table below.

Arch	LM Type	Model	Architecture	Dimensions	Transformer Layers (Blocks)	Attention Heads	Case Sensitivity	Training Dataset	Training Objective	Best Use Cases
Transformer-Encoder	BERT	bert-base-cased	Transformer-Encoder	768	12	12	Case-sensitive	BooksCorpus + Wikipedia	Masked Language Model (MLM) + Next Sentence Prediction (NSP)	Text classification, sentiment analysis, NER
Transformer-Encoder	RoBERTa	roberta-base	Transformer-Encoder	768	12	12	Case-insensitive	Common Crawl (160GB)	Masked Language Model (no NSP)	Long-text understanding, summarization
Transformer-Encoder	DistilBERT	distilbert-base-cased	Transformer-Encoder	768	6	12	Case-sensitive	Distilled from BERT base	MLM (with distillation)	Lightweight Q&A, mobile applications
Permutation Transformer	XLNet	xlnet-base-cased	Permutation Transformer	768	12	12	Case-sensitive	BooksCorpus + Wikipedia	Permutation-based Training (Autoregressive, but sees bidirectional context)	Text classification, document ranking
Transformer-Decoder (Causal Autoregressive)	GPT-2	gpt2	Transformer-Decoder	768	12	12	Case-insensitive	WebText (45GB web corpus)	Causal Language Modeling (strict left-to-right)	Text generation, conversational AI
Transformer-Decoder (Causal Autoregressive)	LLaMA	LLaMA 3.2-1B	Transformer-Decoder	768	12	12	Case-sensitive	Common Crawl + academic corpora	Causal Language Modeling	Conversational AI, multilingual tasks
Transformer-Decoder (Causal Autoregressive)	DeepSeek-Qwen	DeepSeek-R1-Distill-Qwen-1.5B	Transformer-Decoder	1,536	24	16	Unknown	Fine-tuned Qwen base models	Causal Language Modeling	Multi-modal AI, summarization, retrieval
Static Word Embeddings	GloVe	840B.300d	None (Word Vectors)	300	N/A	N/A	Case-insensitive	Common Crawl (840B tokens)	Co-occurrence Matrix Factorization	Word similarity, information retrieval
Static Word Embeddings	FastText	common-crawl 300d	None (Word Vectors)	300	N/A	N/A	Case-insensitive	Common Crawl (600B tokens)	Subword-based embeddings	OOV handling, text classification
Static Word Embeddings	Word2Vec	GoogleNews 300d	None (Word Vectors)	300	N/A	N/A	Case-insensitive	Google News (100B words)	Continuous Bag of Words (CBOW) / Skip-gram	Word similarity, clustering

Figure 2: Language Model Summary Table

5.2 Supported Datasets

For each of the models, we prepare the underlying dataset as needed, create the representation form of the dataset - which is represented as lists or arrays of documents in said representation form – and then we prepare the model (classifier) with the underlying dataset representation, build the model using training

and validation data, and then test the model and report results back based upon the model performance against the test data that we set aside – following classical ML and DL model building and testing procedures effectively.

The datasets we use for testing come from a variety of sources, covering news and academic content as well as medical data and reviews.[16] We look at both single-label classification datasets, the simpler form where a document in a dataset is classified to a single category, and then we look at multi-label classification datasets where a document can belong to multiple categories. We also have one binary classification dataset (IMDB) which is affectively a sentiment analysis labelling scenario.

A brief description of each dataset we used in our different test scripts is provided below.

5.2.1 Reuters21578: multi-label, 115 classes

The Reuters-21578 dataset is a cornerstone in text categorization research, comprising 21,578 news documents from the Reuters newswire in 1987 originally collected and labeled by Carnegie Group, Inc. and Reuters, Ltd. The dataset is multi-label and each document is labeled across 115 different categories, and although not very well balanced across categories, nonetheless it is one of the standard benchmarks for multi-label text classification tasks. Researchers have extensively utilized this dataset to evaluate various machine learning and natural language processing algorithms.

5.2.2 BBC News: single-label, 5 classes

The BBC News dataset contains 2,225 news articles from the BBC website, categorized into five distinct topics: business, entertainment, politics, sport, and tech. This single-label, balanced dataset is particularly useful for testing text classification models given that it is relatively small in size, single-label and well balanced across labels.

5.2.3 RCV1-V2: multi-label, 103 classes

The Reuters Corpus Volume I (RCV1-v2) is an extensive dataset comprising over 800,000 newswire stories from Reuters, covering the period between August 20, 1996, and August 19, 1997. This multi-label dataset spans 103 categories, providing a rich resource for large-scale text classification and clustering research. Its size and complexity make it ideal for evaluating algorithms that need to handle large, real-world datasets with overlapping categories. It is news data and there is a lot of overlap between classes as one might expect from a news site.

5.2.4 20 Newsgroups: single-label, 20 classes

[16] Details on the different in scope datasets can be found in the Appendix

The 20 Newsgroups dataset is a collection of approximately 20,000 newsgroup documents, partitioned across 20 different newsgroups. Each document is labeled with a single category, and the dataset is well-balanced, making it ideal for testing out different classifiers and as such has been a popular benchmark dataset for evaluating text classification algorithms.

5.2.5 arXiv: multi-label, 58 classes

The arXiv dataset consists of research papers from the arXiv repository, categorized into various scientific disciplines. The dataset is multi-label, articles can belong to more than one class (but at least one), and there are 58 class labels. Given the nature of the content, there are many scientific words or phrases (and some special characters) that must be managed to yield good model performance results and as such makes it a good test dataset to evaluate how well models handle domain-specific terminology and semantics.

5.2.6 arXiv protoformer: single-label, 10 classes

A derived or curated version of the arXiv dataset, often used in experiments with prototype-based models (e.g., Protoformer). The set we use is filtered for the most common classes (just 10) and is single label. This is a simplified variant of the full arXiv dataset, which is focused on distinct topics, avoiding overlapping categories.

5.2.7 OHSUMED: multi-label, 23 classes

The OHSUMED dataset consists of medical abstracts from the Medline database, labeled with MeSH (Medical Subject Headings) terms. It is a multi-label test set, consisting of 23 distinct classes. It is focused on medical topics, making it highly domain-specific, meaning that many of the words and terms would not be found in a typical language model training set. The specialized vocabulary presents challenges for text classification models, particularly in handling out-of-vocabulary terms and understanding complex medical jargon.

5.2.8 IMDB: single-label, 2 classes

The IMDB dataset is one of the most common datasets used in Machine Learning and Deep learning research, consisting of movie reviews from IMDB, each labeled with the sentiment (positive or negative) expressed in the review, making it a binary classification problem effectively. It is a well-balanced dataset with 25,000 training samples and 25,000 test samples. Text length varies, with some reviews being long and detailed.

5.2.9 Dataset Summary Table

A summary table of the different datasets we use with Layer Cake and their basic characteristics is provided below.

Dataset	Type	Classes	Size	Characteristics
Reuters-21578	multi-label	115	64 MB	News articles, imbalanced class distribution.
BBC News	single-label	5	6.7 MB	Balanced topics from BBC news.
arXiv	multi-label	58	5.5 GB	Research papers, overlapping categories.
arXiv Protoformer	single-label	10	147 MB	Simplified version of arXiv for single-label tasks.
IMDB	single-label	2	694 MB	Movie reviews, sentiment analysis.
OHSUMED	multi-label	23	387 MB	Medical abstracts, domain-specific.
RCV1-v2	multi-label	101	7.4 GB	Large-scale news corpus, hierarchical topics.
20 Newsgroups	single-label	20	15 MB	Newsgroup articles, balanced and diverse topics.

5.3 Supported Metrics

Primarily we look at *macro-f1* and *micro-f1* scores for the different classifier, language model, and representation form test runs, but we also track *precision*, *recall*, *accuracy*, *Hamming Loss* and *Jaccard Index* numbers for each model run as well, although these latter metrics are not used in our analysis.

Note that we use the *macro-f1* score to determine whether or not our neural models are improving and to determine whether or not we have met our early-stop condition. We briefly cover how these metrics are calculated below. As a refresher, we outline how we compute these metrics within the context of Layer Cake benchmarking.

Precision measures the accuracy of positive predictions. It is the ratio of correctly predicted positive observations to the total predicted positives, or

$$\text{Precision} = \frac{TP}{TP + FP}$$

where *TP* is true positives and *FN* is false negatives.

Recall (sensitivity) on the other hand, measures the ability of a model to find all the relevant cases (i.e., True Positives) within a dataset. It is the ratio of correctly predicted positive observations to all observations in the actual class, or mathematically:

$$\text{Recall} = \frac{TP}{TP + FN}$$

The *F1 score* is a more widely used measure of predictive performance given that incorporates precision and recall into one measure using the *harmonic mean*, defined mathematically as:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The macro variant of the F1 score, i.e. *macro-f1*, uses the *arithmetic mean* of all the per-class F1 scores, treating all classes equally regardless of their *support* - i.,e. the number of times the class is found in the training data set. This unweighted average F1 Score across classes is defined mathematically as:

$$\text{Macro-F1} = \frac{1}{N}\sum_{i=1}^{N} 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

Where Precision and Recall are calculated as outlined above and *N* is the number of classes.

The *micro-F1 Score* on the other hand uses the class specific contribution to compute the average F1 score, measuring the aggregate contributions of all classes with respect to the average score, or mathematically:

$$\text{Micro-F1} = 2 \times \frac{\text{Micro-Precision} \times \text{Micro-Recall}}{\text{Micro-Precision} + \text{Micro-Recall}}$$

Where Micro-Precision and Micro-Recall are the precision and recall calculated globally.

5.4 Language Representation and Test Workflow

The baseline ML classifiers we use within the context of this work are Support Vector Machines, Logistic Regression and Naïve Bayes models, the first two of which effectively work by optimally separated the underlying dataset representation, in high dimensional space defined by the space of words in the underlying dataset. The Naïve Bayes model is a pure probabilistic approach and while its use is limited, due to the constraint that it must work in the positive real number space, it nonetheless provides a nice probabilistic baseline for our analysis.

The neural models come in two forms - the neural models that are designed to work with static word embedding language models such as GloVe and Word2Vec (and FastText), and then the transformer architecture models which are much more sophisticated and computationally intensive than their static model counterparts. For these latter models we leverage the Hugging Face APIs which not only provide the ability to work with the various models that are included in this study but also come with predesigned Sequence Classifier models that are designed specifically for sequence, i.e. text or natural language, classification.

5.4.1 Machine Learning Approaches (Baseline Testing)

With our baseline testing, we experiment on a variety of forms of language representation given that the model design is straightforward enough, and flexible enough, to support a variety of different language representation methods. These models are constructed using the base python sklearn libraries and as such they are CPU bound, as opposed to the GPU bound neural models that we describe below. While we primarily look at these baseline ML classifiers as baseline test cases to be viewed against the backdrop of their more sophisticated – and computationally intensive – counterparts, these models are nonetheless effective in and of themselves as standalone text classification solutions and are very easy to work with and perform very well on small and medium size datasets.

Another insight that is revealed within the context of this research is that with the static embedding (language) models, one has a great deal of flexibility with respect to how the embedding layer, which is the storehouse of the word-embedding matrix basically, is setup and initialized. The fundamental method that we use for our tests, again following (Moreo, et al., 2021), is to set up the embedding layer with only the necessary and requisite embeddings in the dataset that we are solving for – effectively reducing the semantic embedding space by the difference between the language model dictionary size and the dataset dictionary size (as measured by the language model in question). This computational optimization is significant, as our tests show, and these hand rolled PyTorch models are very efficient, albeit not quite as effective as their transformer cousins, even with the additional assistance from WCEs.

The transformer models however, given that they aren't as much embedding storehouses as much as they are systems of embedding generators, have a much more tightly coupled and rigid structure that does not lend itself to customization at all really. This of course not only has performance implications given that invariably the language model supports tokens that are not relevant for a given dataset we may be fine-tuning the model on, but nonetheless are loaded into the model memory during training and testing adding dimensional complexity to each and every operation, and greatly increasing the amount of data that needs to be synchronized across GPUs for large parallel model training which in fact is the norm when it comes to LLMs given their size. This undoubtedly is a factor in the results around model construction times where the transformers, as expected, way exceed the training times for both the ML and static LM neural models.

We have two basic forms of language representation that we test when working with the baseline ML in models that we support with Layer Cake, each with its own computational architecture essentially and each coupled with its own forms of feature reduction, a critical aspect of model optimization in the ML world given the dreaded *curse of dimensionality*. The two basic forms of language representation are:

- **i.** *vectorization forms*: Layer Cake supports both Count and TF-IDF Vectorization with built in feature reduction using the sklearn min_df feature which only includes words, or tokens, that appear in more than min_df docs, in our case 5. We call these language representation forms *vmode* in our analysis and they come in two basic flavors, Count and TF-IDF, [17]
- **ii.** *embeddings*: word, or again token, level representations of real-number vectors of fixed dimensions (set by the underlying language model during training), which also come in two basic flavors, static and dynamic.

Each of these representation form types is tested with and without Word-Class Embeddings[18], given that we have the ability to perform various forms of linear algebraic transformation on the primary language representation form and as long as we end up with a feature matrix (a dictionary size by feature size matrix essentially) that aligns with the label matrix, the model will classify the data fairly consistently and effectively. The setup for the neural model is much more rigid, with a language model token indexing strategy for the

[17] The TFIDFVectorization arguments that we use, following the 2019 study, are *min_df=5* which indicates that a word, or token, is only included in the doc vector if it exists in at least 5 documents across the dataset, and *sublinear=True* which applies sublinear scaling to the tf-idf values that are used in the underlying doc vector representation, i.e. the tf value is replaced with 1+log(tf), see https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html for details. Note also that while the Layer Cake platform supports Count vectorization as well, these results are not reported in this study as they are what one might expect, that the tf-idf representation performs better generally than the straight up count vectorization, i.e. bag of words model.

[18] As defined in the study of the same name published in 2021 (Moreo, et al., 2021) which forms the basis of the foundational code for Layer Cake.

embedding layer effectively – optimized in the static case against the dataset we want to tune the model to, and fully loaded in the case of the transformer models.

We also include a few variants of Latent Semantic Analysis (LSA), sometimes called Latent Semantic Indexing (LSI), into the ML representation form mix as well, a method that continues to show its value as an extremely effective model optimization technique. We even show that LSA can be used in conjunction with WCEs / TCEs to boost model effectiveness, consistent with the results found by (Moreo, et al., 2021) with respect to WCEs boosting the performance of DL approaches with static embeddings. LSA uses a matrix factorization technique called *singular value decomposition* (SVD) which generalizes the eigendecomposition of square matrices to non-square matrices.
One of the advantages of looking at these different representation forms within the context of these classic machine learning classifiers is that the underlying mathematics, and the underlying forms of language representation are relatively straightforward and are coupled together directly, as a form of linear separation in vector space, in semantic space really following (Worth, 2023). Once we get to the transformer classifiers, we become constrained by the underlying language model itself in terms of how the text can be represented. We discuss this more below in the section on dynamic, contextualized deep learning classifiers, aka *transformers*.

The ML framework in fact, is flexible enough to not just support different text vectorization techniques and different embedding structures along with LSA as a form of feature reduction, but it also can be used to test a wide variety of different combinations thereof, including Word/Token Class embedding integrations into the feature mix as well, allowing for insights into the performance of the various forms of language representation both in terms of effectiveness (*macro-f1* and *micro-f1* statistics) as well as model training times, i.e. *timelapse analysis*. These various forms of (linear algebraic) transformation amount to, in the case of feature reduction, semantic spatial space compression where a higher dimensional language representation form is projected into a lower dimensional form using matrix dot product operations. This of course in many respects what language models do with their embedding structures, reduce the feature space for the word or token representation from the dictionary size down to the embedding vector dimensional size (the *solo* and *doc* representation forms described below).

The table below summarizes the various forms of language representation that are used within the context of the ML baseline testing of Layer Cake.

Representation	Description	Feature Sources	Key Use Case
solo	Embedding representation alone	Embeddings (weighted/avg/summary)	When embeddings alone are sufficient
solo-wce	Embeddings + Weighted Class Embeddings	Embeddings + WCEs/TCEs	When class interaction features improve results
cat-doc	Vectorized text + Embedding representation	TF-IDF/Count + Embeddings	When combining raw text and embeddings enhances performance
cat-wce	Vectorized text + Weighted Class Embeddings	TF-IDF/Count + WCEs/TCEs	When integrating raw text with class interaction features is needed
cat-doc-wce	Vectorized text + Embedding representation + Weighted Class Embeddings	TF-IDF/Count + Embeddings + WCEs/TCEs	Comprehensive representation combining raw, embedding, and class features

Representation	Description	Feature Sources	Key Use Case
dot	Projection into embedding space via dot product	TF-IDF/Count + Embedding Space	When projecting text into embedding space is crucial
dot-wce	Projection into embedding space with Weighted Class Embeddings	TF-IDF/Count + Embeddings + WCEs/TCEs	Projection combined with class-sensitive embedding
lsa	Latent Semantic Analysis for dimensionality reduction	TF-IDF/Count + LSA	For dimensionality reduction while keeping semantic content
lsa-wce	LSA + Weighted Class Embeddings	TF-IDF/Count + LSA + WCEs/TCEs	When reduced features and class sensitivity are both needed
vmode	Raw vectorized representation (TF-IDF or Count)	TF-IDF/Count	Baseline approach using only vectorized data

The basic workflow of the ML classification test module follows the basic steps of any standard NLP workflow – data preparation, model training, model testing. We simply scale the problem using all of the datasets, language models and other configuration parameters (hyperparameters) to drive each of the test runs, with the output logged that captures all of necessary and poignant information about the test run – including but not limited to: classifier type, model hyperparameters, system info, language model, metric, value, timelapse, etc., the basis for our analysis. Layer Cake includes a reporting and analytics module which generates charts, heatmaps and summary data related to each and every test run, logging, macro-f1, mico-f1, accuracy, precision, recall, hamming loss and jacard-index values for every model configuration.

Our test script loops through all the different in scope Classifiers, along with the in scope hyperparameters and representation forms, prepare the data as needed for the model in question, build and train the model with the training and validation data from the dataset (supervised), and then test the model against the test data we set aside when we initialized the dataset docs. For the multi-class datasets, we use a one-vs-all approach, encapsulated in the OneVsRestClassifier API from sklearn which involves fitting one classifier per class and then for each classifier the class is fitted against all other classes.[19] For single-label datasets, Layer Cake uses the relevant classifier LinearSVC, LogisticRegression, or MultinomialNB as the case may be, again we leverage the sklearn APIs for these classifiers as well. The test data we look at below uses the default model parameters.

In preparing the data, we preprocess it by retrieving the data from the relevant Layer Cakes datasets subdirectory, downloading it if necessary, and then for the word and sub-word based language models (i.e. GloVe, Word2Vec and fastText) we lowercase the text, remove punctuation, and remove stopwords whereas for the token based, transformer models (GPT2, BERT, DistilBERT, DeepSeek, RoBERTa and XLNet) we leave the underlying text intact as the transformer based models are designed to handle full sentences and given their dynamic, contextualized computational nature the stopwords and case sensitivity allows the models to generate better embeddings. We then construct the label matrices from the training and validation data in the right form depending upon whether or not the underlying dataset is single-label classification or multi-label classification, and then proceed to compute, and cache, the various base representation forms of the text which include vectorization forms as well as the embedding dataset dictionary and document encoding structures to be used by the underlying classifiers for the different. These operations can be computationally expensive, so we do them once for each dataset, language model and representation form and then use these cached files, pickle files, throughout.

[19] See https://scikit-learn.org/0.24/modules/generated/sklearn.multiclass.OneVsRestClassifier.html

The vectorized text is constructed using TFIDFVectorizer API from sklearn, which is CPU bound (as opposed to GPU bound as the deep learning classifiers are, requiring special hardware effectively), which is called a) with the min_df=5 flag which operates as a form of feature reduction by limiting the words, subwords or tokens that are included in the output tf-idf vectors to those tokens that are found in at least 5 documents (called 'cutoff' in the literature) and b) we use the sublinear argument to log normalize the tf-idf values, converting the term frequency (*tf*) value to 1 + log(*tf*).[20] The other representation forms include the *embedding_vocab_matrix* structure which is a static representation of the dictionary of words or tokens that are in scope for the given dataset computed from the relevant underlying language model, and then the dataset representation in the underlying language model itself which is done via the *encode_docs* method that we describe above.

Note that for the transformer models, the document encoding is computed dynamically, as the documents are passed through the classifier, via the transformer model itself which sits at the forefront of that computational edifice so to speak. For the static language models the representation is well, static, a simple mapping exercise for each word, or n-gram, in the dataset is represented by a single static embedding representation. The computational component here is how it is that these transformer models get more accurate in the construction of their semantic space, and also where a good deal of the computational overhead is.[21]

5.4.2 Neural Models (Deep Learning)

The neural classifiers we use in Layer Cake are split into two groups, one for the static word embedding language models (Word2Vec, GloVe and FastText) and another for the transformer models, each tested under various hyperparameter conditions and each with its own distinct, although closely related, form of language representation. In fact, technically, each language model has its own form of language representation, albeit all architecturally analogous, all nonetheless distinct with respect to how the underlying feature space – semantic space – is defined. Part of what we are looking to understand with Layer Cake in fact is the difference this type of dimensional attribute has on classifier performance, if any.

The first neural model test script is inherited from (Moreo, et al., 2021) but extended to support Word2Vec and FastText in addition to GloVe. The script, the dataset preparation, and the classifier design in this module are custom tailored, and optimized, to fit the static language models. The classifiers are hand crafted PyTorch models that are set up in very simple CNN, LSTM and ATTN (attention) based neural models, basically sequence classifiers. The second test script we use is for the transformer language models, which require slightly different text preprocessing (we leave the text in its raw form basically) and is built using the HuggingFace (HF) libraries which come with a built basic form of parallelism which is convenient for testing.[22]

---

[20] This approach follows (Moreo, et al., 2021). For details on TFIDFVectorizer API from sklearn see https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

[21] In the optimized form, denoted as '[opt]' in the representation shorthand we use throughout, we use a random sampling of grid search options approach via the sklearn RandomizedGridSearchCV API, using 9 as the n_iter argument (9 options are tried), the technique being a form of optimized way to get close to the optimal set of values for a given model configuration. It's worth noting however that the variance in terms of model performance between these optimized models and the default parameter ([def] representation form) models is not as significant as one might think and the performance hit for building and testing the optimized classifiers is, at least for the ML test cases, significant.

[22] As of this writing Hugging Face supports over 300,000 language models.

Just as with the ML classifiers, the neural testing covers both single label classification datasets, where each document gets classified to one and only one class label, as well as multi-label datasets where a document can be classified to one or more labels or classes. Special handling must be done to ensure that our neural classifiers support both cases, just as in the ML case, and this is accomplished by ensuring that the format of the labels that is fed into the classifier is correct and reflects the dataset type, and the classifier itself is set up with the proper loss function depending on the dataset type as well – CrossEntropyLoss for single-label datasets and BCEWithLogitsLoss for multi-label classification.

The static word embedding (neural) classifiers come in three neural variants, simple PyTorch based CNN, ATTN and LSTM architectures, tested in various hyperparameter configurations with and without the WCEs, and our transformer language model classifiers come in two variants, a HF Sequence Classifier variant (hf.sc) which encapsulates the classifier logic for you for each language model, as well as simple CNN classifier which is modeled after its neural counterpart for comparison (hf.cnn). What follows below is a brief description of the test workflows for each of the static neural classifier testing as well as the transformer model testing, with preliminary analysis of each set of data individually, and then combined together so that we get a clear picture of classifier and language model performance in each setting, with again a focus on macro-f1 and micro-f1 metrics, along with training times.

5.4.2.1 Static Neural Classifiers

We generally follow the same test, data preparation, model training and testing workflow that we used when we tested the baseline machine learning classifiers, except that now given that we are working with a neural model, we have very specific requirements with respect to how we are to represent the sequential text data such that the neural model can train effectively. Essentially this involves, as discussed earlier, building a dictionary for the underlying dataset that is a function of the language model (and tokenization strategy) we are working with, and then indexing the dataset against this dictionary and then using these static embeddings at runtime (in the forward method of the neural model training) to represent the text (sequence) which is fed into the classifier head.

For the static neural models, we first initialize and prepare the underlying dataset, and, from a text preprocessing perspective, consistent with best practices for static language representations, we remove stopwords and punctuation from the text before we index it for input into the neural model. [23] The text is then converted into an index representation form which is typical for NLP deep learning architectures, where each word is indexed into a dictionary that maps back to the associated index in the underlying language model, facilitating a lookup of the embedding for the word in question during the forward pass of training. For the transformer based classifiers however, we leave the text as is for the most part, allowing the transformer model tokenizers to do their thing and capture as much information about the underlying text as possible, something that distinguishes the transformer models from their static embedding counterparts which use word, or subword (n-gram) tokenization strategies and generally are not designed to handle

[23] This is in contrast to the data preparation for the transformer based models where we keep the text case sensitive, and leave the stopwords and punctuation in place given that the transformer based models are designed to handle the full complement of language syntax, one of its prominent features that distinguishes it from its static language model counterparts which are not, by design, 'context aware'.

punctuation. Computation of the different language representation forms we use for the various tests we perform, is expensive, so we cache this information using pickle files for later use.

The language representation form that is used is to construct the neural model classifier embedding layer for the dataset in question using a) the dataset vocabulary words (post vectorization (i.e. vectorized using *TFIDFVectorizer* with 'cutoff' = 5 so that this dictionary is a subset of the entire dataset dictionary) combined with the static embeddings for all *known* words (i.e. words or terms that have defined static embedding representation as defined by the underlying language model, or in other words all words that are not in the vectorized dataset representation but exist in the underlying dataset doc set and have a corresponding definition in the underlying language model), together forming what you might call the static embedding layer dictionary.

The input text data is tokenized into individual words using the dataset's vectorization strategy (e.g., TF-IDF or count vectorization) and each token is mapped to its corresponding embedding using the custom embedding layer, which is designed to support both pretrained embeddings, learnable embeddings, as well as supervised embeddings, i.e. WCEs, which serve to extend the pretrained language model representation of a given word using class, or label, co-occurrence statistics, serving to add additional information to the pretrained language model representation of the word and extending the dimensions of the underlying word vector by the number of classes in the dataset. The indexed document representation form, which maps to the pretrained underlying word embeddings, is standard (best) practice for neural text classifiers as it allows for efficient lookup of token embeddings from the embedding matrix. Note that in the static neural model classification design, we also add special tokens for unknown words (UNKTOKEN), and padding (PADTOKEN), which are leveraged by the model to ensure the neural model handles the text representation form correctly for classification, meaning that all OOV terms are treated by the model in the same way as a special token, with its own embedding representation.

For example, if we look at the static embedding CNN neural architecture for example in the figure below, it is the EmbeddingCustom layer which encodes the (static) embeddings for the entire vocabulary that is in scope for the dataset in question. Given that the neural model is 'hand-crafted' so to speak, we populate this embedding layer only with words (or subwords in the case of fastText) that exist in the dataset we are classifying, an optimization that we do not have the flexibility to implement with the dynamic, transformer models in fact and one of the reasons that these static models are so much more efficient computationally than their dynamic counterparts. The custom nature of the neural model allows us to change the dimensions of the underlying EmbeddingCustom layer as well - like for example when we add WCEs into the semantic space which we do by concatenating these embeddings, whose size is the number of class labels, to the language model embedding layer directly, allowing the model to learn the optimal set of weights from the entire embedding set from the start.

```
NeuralClassifier(
  (embed): EmbeddingCustom(
    (pretrained_embeddings): Embedding(22846, 300)
  )
  (projection): CNNprojection(
    (conv1): Conv2d(1, 128, kernel_size=(3, 300), stride=(1, 1))
    (conv2): Conv2d(1, 128, kernel_size=(5, 300), stride=(1, 1))
    (conv3): Conv2d(1, 128, kernel_size=(7, 300), stride=(1, 1))
    (dropout): Dropout(p=0.5, inplace=False)
  )
  (label): Linear(in_features=384, out_features=5, bias=True)
)
```

Figure 3: Static Language Model Classifier (CNN Architecture)

The custom embedding layer combines pretrained embeddings with learnable embeddings, and optionally WCEs, to allow the model to be tailored specifically for the dataset in question that is fed into the model for training. This representation form is what is used to initialize the embedding layer of the neural models, which again is used at runtime, i.e. at training time, to be retrieved by the classifier and processed by the neural model in question, pooled together in batches, and ultimately classified the transformation layers of the different deep learning models we test with.

Once the dataset is indexed, a process which must be aligned with the tokenization strategy of the underlying language model we are working with, we then test each of the static word embedding neural classifiers under different hyperparameter settings and with each of the in scope datasets, both with and without WCEs, resulting in metrics – like the initial set of ML classifier data – that can be directed compared against the 2021 study. During training, these indexed, numerical representations of the documents are batched and passed to the model's embedding layer through the NeuralClassifier so that, via the EmbeddingCustom layer, the appropriate embeddings for each token from the embedding matrix (pretrained + learnable embeddings) can be retrieved and used for downstream processing.

The embeddings are then, in the forward method of the classifier, processed by either the CNN, LSTM, or ATTN projection layers depending upon the model being used, the end result of which yields a contextual, language model embedding representation of the documents is created for the underlying model. One of the primary differences between the static language models and the dynamic ones in fact, is the tokenization strategy that is used. While static word embeddings are designed to look at, and represent words[24], the transformer-based models break language down into tokens, which while solving for out of vocabulary and polysemy type problems, nonetheless moves the language representation form away from semantics – what something means - to something more basic and fundamental, and certainly more abstract.

5.4.2.2 Dynamic Neural Classifiers (transformers)

The transformer model testing workflow, and data preparation, is slightly different than the neural classifier test script for the static pretrained language models. We use two distinct classifier architectures that are built off of the Hugging Face libraries - namely the SequenceClassifier and a simple CNN, neural network architecture that is modelled after the same neural architecture we use for the static language model testing. The basic test workflow is designed to generate a Classifier of a given type, with a given pretrained language model, a given underlying dataset, and a set of program and model hyperparameters that define the test run. Each run is logged with details around system parameters (OS, GPUs, CPUs, memory, etc.), language model type and attributes, the dataset and classifier type, and other hyperparameters to be used by Layer Cake Reporting & Analytics module for analysis.

The power of this transformer-based approach, which dominates the language model research and development landscape right now given how effective they have proven to be, is predicated on what might be called its 'specificity', or 'granularity', in embedding creation, also results in constraints with respect to not only forms of language representation that will work with, or are compatible with, the underlying model,

[24] Or in the case of fastText subwords but ultimately these are words in the sense that the subwords themselves do in fact have a semantic meaning

but also in terms of (perhaps most profoundly) system resource and computational requirements which are necessary to generate the kind of granularity necessary to drive the performance improvements. Much of the computational complexity around working with these models is from this runtime complexity that is driven from the computational requirements of the underlying language model itself, which clearly does a lot of work to generate the context aware embeddings, which are of course the main distinguishing characteristic of the transformer-based models versus their static counterparts. In other words, given the dynamic nature of the transformer based embeddings, the embeddings – the model really – has to sit in a different type of neural classifier entirely that accounts for, and incorporates, the underlying language model directly into the classifier so that the embeddings can be computed, dynamically, and then they can be pooled and fed to the classifier head portion of the model.

One of the implications of this technical dependency is that the static based neural models are not only significantly less computationally intensive than their transformer based counterparts (one of the key measurement findings of this study), but also more flexible in terms of in terms of how the embedding layer can be structured – the embedding layer being the foundational structure of all of the neural model classifiers that is used to generate the embeddings that are used to represent the text of the underlying dataset, either as a straight index lookup in the forward pass of the model as is the case with the static neural models, or as inputs into the runtime computational dynamic embeddings that the transformer models generate and that can be consumed, and again pooled, for classification in the forward pass of model training.[25]

The transformer-based classifiers are - given the dynamic nature of the underlying language models – integrated directly with the language models which are neural network models in and of themselves, each of which dictates the structure, size and content of its underlying embedding layer - i.e. the specific model data. Outside of the computational logic differences between the static and dynamic language representation forms, the other main difference between the two sets of neural classifiers is of course both the structure, and content, of this embedding layer which provides the computational basis for embedding representation by the language model for the dataset in question – for both the static language models as well as the dynamic ones. As illustrated in the CNN HF transformer classifier shown below for example, the embedding layer is initialized to include the entire dictionary of token embeddings for the *bert-base-cased* language model we use, a language model which has 28996 pretrained token embeddings, each of which is of size 768, i.e. the embedding dimension size or the *hidden_size* of the (language) model.

[25] Effectively the transformer-based models, given their internal dependency on the underlying computational logic that is intrinsic to the language model type itself, cannot be extended horizontally (concatenated) with TCEs as is the case with the static models.

```
LCCNNBERTClassifier(
  (loss_fn): CrossEntropyLoss()
  (pretrained_embeddings): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(28996, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSdpaSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (conv1): Conv2d(1, 128, kernel_size=(3, 768), stride=(1, 1))
  (conv2): Conv2d(1, 128, kernel_size=(5, 768), stride=(1, 1))
  (conv3): Conv2d(1, 128, kernel_size=(7, 768), stride=(1, 1))
  (dropout): Dropout(p=0.5, inplace=False)
  (classifier): Linear(in_features=384, out_features=5, bias=True)
)
```

Figure 4: LCCNNBERTClassifier Architecture (transformers)

The HF CNN architecture, as the name implies, leverages a CNN neural architectural approach to classification constructed directly on top of the HF transformer model itself, bypassing the HF Sequence Classifier neural models, effectively as a point of comparison against the HF Sequence classifiers themselves. The figure below shows what that neural model looks like and while similar to its SequenceClassifier cousin it is nonetheless distinct and provides a connecting architecture with the static models which also have a CNN architecture test case. With these neural models we once again see the transformer model itself at the top of the stack as it were, followed by three CNN layers, just like the static neural models, and then a final linear transformation layer which maps to the labels of the dataset – 5 in this case indicating that we are working with BBC-News test case (CrossEntropyLoss for single label datasets).

We would point out that for both of these transformer model classifiers that we test with, the embedding layer is fixed, and constrained by the underlying language model itself – which in this case consists of 28,996

token embeddings, each of which is a vector of size 768 and each of which maps back to a token in the dictionary space of the specific BERT language model variant being used - *bert-base-cased* in our work. As mentioned previously, given the tight coupling of this embedding layer with the runtime computational requirements to generate the context aware embeddings, we were not able to extend, or minimize (optimize) this embedding layer at all for any of the transformer models, so it's fair to say that with the extra complexity comes a lack of flexibility with respect to how it is we can set up these models and how, if at all, they can be extended to embed additional information into them – like TCEs for example.

As one might expect, this embeddings layer, *BertEmbeddings* in this case, is tightly coupled with the language model, which as one can see from the neural model shown below actually makes up the bulk of the whole neural model classifier itself. It's this model integration which allows for the dynamically generated (transformer) embeddings, for the specific sequence in question, to be passed to the classifier layers of the neural model during training (the forward pass). In other words, one of the distinguishing characteristics of these transformer models is that they themselves are rigidly fixed and heavily dependent on both the size and dimension, and contents, of their embeddings. While this shouldn't come as a surprise necessarily, it should be kept in mind when one considers what kind of flexibility one needs with a language model – like for example if there are other embedding forms that one wants to use in conjunction with pretrained language model embeddings, TCEs for example.

In other words, given the more rigid nature of the embedding layer for the HF transformer classifiers, they are not well suited for either a) filter optimization methods example to only include token embeddings for those in scope for a given dataset, or b) extend the feature space of the embedding layer to include embeddings trained outside the model, like for example WCEs or TCEs in the case of transformers. Both of these optimizations are available if we are hand crafting the neural classifier using static embeddings however and they are applied for this set of test data (the static embedding test data) and have a significant impact on the runtime performance, aka computational requirements, of these models. This cannot be emphasized enough how important these factors are in any cost-benefit analysis of the relative performance of classifiers and/or language models and/or representation forms, as we demonstrate in this work.

The only way in fact that we could test the relative effect of TCEs in our HF, transformer-based language model driven classifiers was by was by including the TCEs with the dynamically generated embeddings in the forward pass of training and using various combinations of these two types of embeddings into the downstream classifier parts of the neural network architecture. And what we found, quite conclusively after a wide range of approaches to integrating the TCEs with the transformer model generated embeddings was that if anything, the inclusion of TCEs into the transformer model classifiers - whether by concatenation or dot product operations, i.e. linear transformations or projections effectively[26] – made the classifier less effective, despite the additional information that the TCEs contained. This is in fact, another one of the unique contributions of this work, i.e. that not only does adding the TCEs into transformer models prove

[26] We experimented with dot product operations, addition operations, and concatenation operations of the transformer generated embedding representations of the text with the TCEs that we computed per class (per token). None of these methods worked for any of the transformer models we tested. We hypothesize that, given how well the transformer models perform generally relative to both their static embedding counterparts and the baseline ML classifiers we test with (SVMs, Logistic Regression methods primarily), that this is because the additional information that is encapsulated in these TCEs is already part of the underlying transformer model representation of the text at some level and therefore adding it to the embeddings only proves to reduce the effectiveness of the classification portion of the classifier. NB The WCEs are combined with the static language model embeddings via concatenation, i.e. we extend the embedding layer dimensions to include the WCEs yielding a richer word representation that is fed into the model during training.

ineffective, but that the underlying embedding layer of the transformer models does not lend itself to customization – period.

The final results we look at in for the transformer classifiers use a 4 A100 GPU system, distributed using the PyTorch Data Parallel form of model parallelism, which is the default mode used by the HF APIs, whereas the neural models needed just one (dedicated) GPU. For the HF transformer language model script, we have two different classifiers that we use, a built in HF Sequence Classifier which encapsulates the language model specific logic for you in a custom classifier alongside a simple CNN based architecture which is modelled after the CNN neural net that we use with the static embeddings.

6 Test Data Dimensional Analysis

In this section we analyze the data from the different Layer Cake test scripts, focusing on the primary dimensions of analysis that we are concerned with in this study, namely classifier architecture and language representation forms. We look at the ML data first, followed by the neural models, and then the summary analysis across the broad spectrum of classifiers included in Layer Cake to a get a full picture of the data.

For the purposes of the neural model analysis, we group the static and dynamic classifier test results together, yielding a comparative perspective for this specific dimension, static versus dynamic language models, which gives us a unique perspective on a) how much better the transformers are for this particular use case (classification) and b) what the underlying cost is computationally for this improvement. This puts us in a unique position to be able to ascertain the cost, again computationally, for the step up in model improvement – a measure we have not seen in the literature before.

For the ML classifiers, our baseline testing module effectively, we look at the several different dimensions of the results data, evaluating performance across the classifiers, the representation forms and the language model types.[27]

6.1 Baseline ML Results

We look at the baseline ML result data by Classifier, by Dataset, by Representation Form, and by Language Model, each dimension supported by visualizations generated by Layer Cake's Reporting and Analytics module and each described in turn below.

6.1.1 By Classifier

To visualize the results by classifier – SVM, LR and NB models - we use a box plot, aka a box-and-whisker plot, which shows the data distribution based on a five-number summary: *minimum, first quartile (Q1), median, third quartile (Q3), and maximum* which helps show the central tendency of the underlying data as well as the data spread and outliers. The "box", which is the shaded area (from Q1 to Q3) represents the interquartile range (IQR), which is the range between the first quartile (Q1) and the third quartile (Q3). The height of the box gives you an idea of the spread of the middle 50% of the data, i.e. a taller box indicates more variability in the data, while a shorter box means the values are closer together.

[27] Note that all of the data we look at for the ML classifiers was generated with the *default* parameters, to get the breadth and scope of the hyperparameter and representation forms for the different variations of classifiers and language types, optimized runs proved very time consuming, particularly the LR classifiers for the larger datasets (RCV), and language models (Llama).

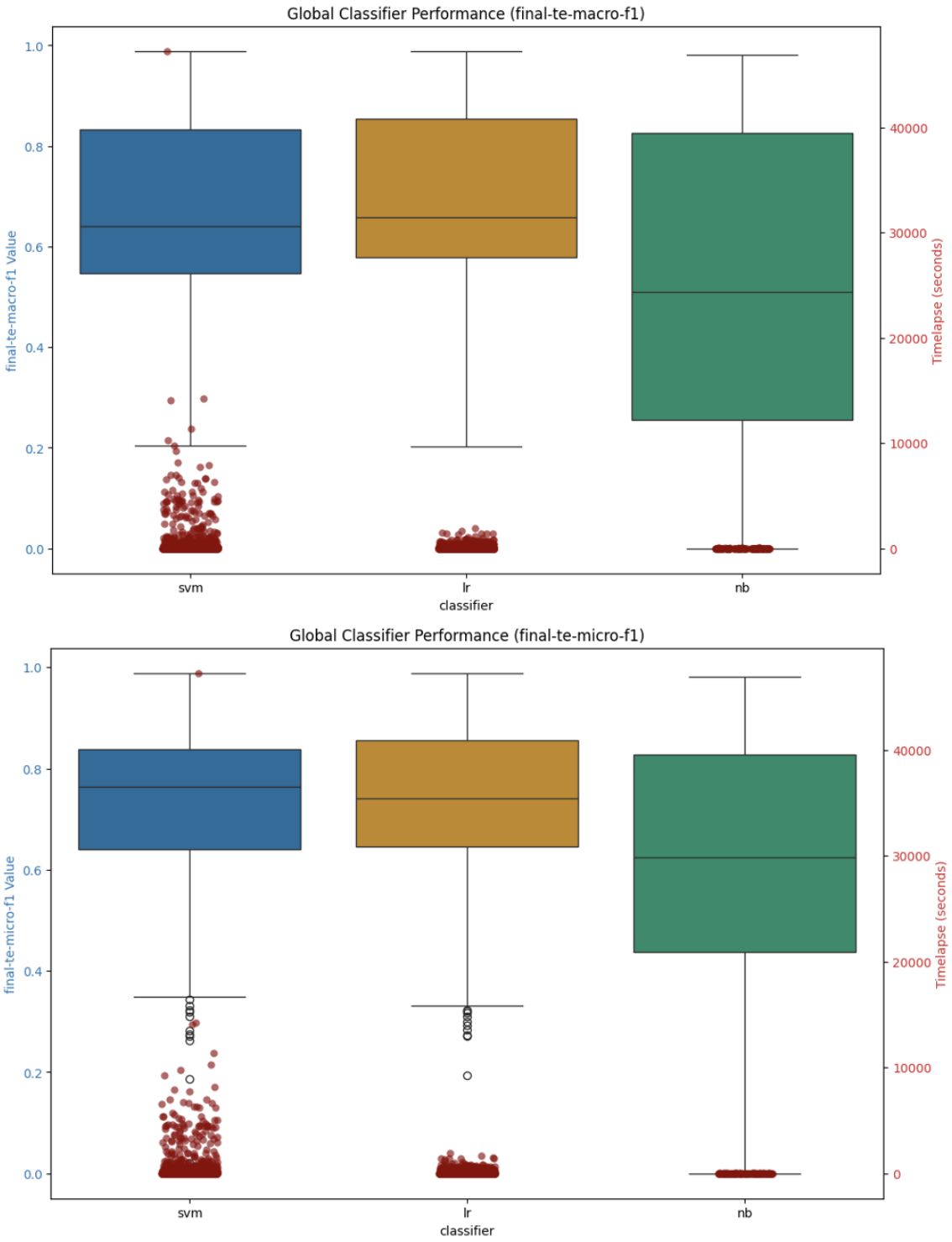


Figure 5: Machine Learning Classifier Performance Summary

On average across the datasets and language models, the Logistic Regression (LR) models perform slightly better than the SVM classifiers, and both the SVM and LR classifiers perform slightly better than the Naïve

Bayes models.[28] The runtimes for the classifiers are clustered below 10,000 seconds, which equates to just under three hours for model construction, with the bulk of the classifiers, particularly for the Logistic Regression models, at around 1000 seconds which is 16 minutes, very reasonable.

The SVM classifier models take slightly more time to build which is expected given the computational complexity of the underlying mathematics for SVMs versus their LR or NB counterparts. Naïve Bayes is the most efficient of the models but given the nature of the model does not support negative numbers and as such can only be used with text vectorization representation forms, most embeddings spill into negative dimensional space, but generally speaking the metrics hold up for NB as well.

6.1.2 By Dataset

When we look at this data by dataset, we see the SVM model performing slightly better for the arxiv data, but for the other models the LR classifiers perform slightly better.

Macro-F1 Detail

Dataset	Type	# labels	LR		SVM		NB	
			Value	Time	Value	Time	Value	Time
20newsgroups	single-label	20	0.617	72.27	0.590	56.74	0.586	11.70
arxiv	multi-label	58	0.616	116.04	0.616	74.89	0.234	9.24
arxiv_protoformer	single-label	10	0.802	173.36	0.810	946.06	#DIV/0!	#DIV/0!
bbc-news	single-label	5	0.968	3.24	0.953	3.10	0.968	0.09
imdb	single-label	2	0.858	132.26	0.846	84.66	0.821	4.90
ohsumed	multi-label	23	0.574	198.28	0.568	558.33	0.309	13.83
rcv1	multi-label	101	#DIV/0!	#DIV/0!	0.489	2639.73	#DIV/0!	#DIV/0!
reuters21578	multi-label	115	0.506	64.73	0.505	107.89	0.290	11.78
Summary								
Logistic Regression							0.701	107.70
SVM							0.674	557.52
Naïve Bayes							0.535	8.59

Micro-F1 Detail

Dataset	Type	# labels	LR		SVM		NB	
			Value	Time	Value	Time	Value	Time
20newsgroups	single-label	20	0.625	72.27	0.596	56.74	0.610	11.70

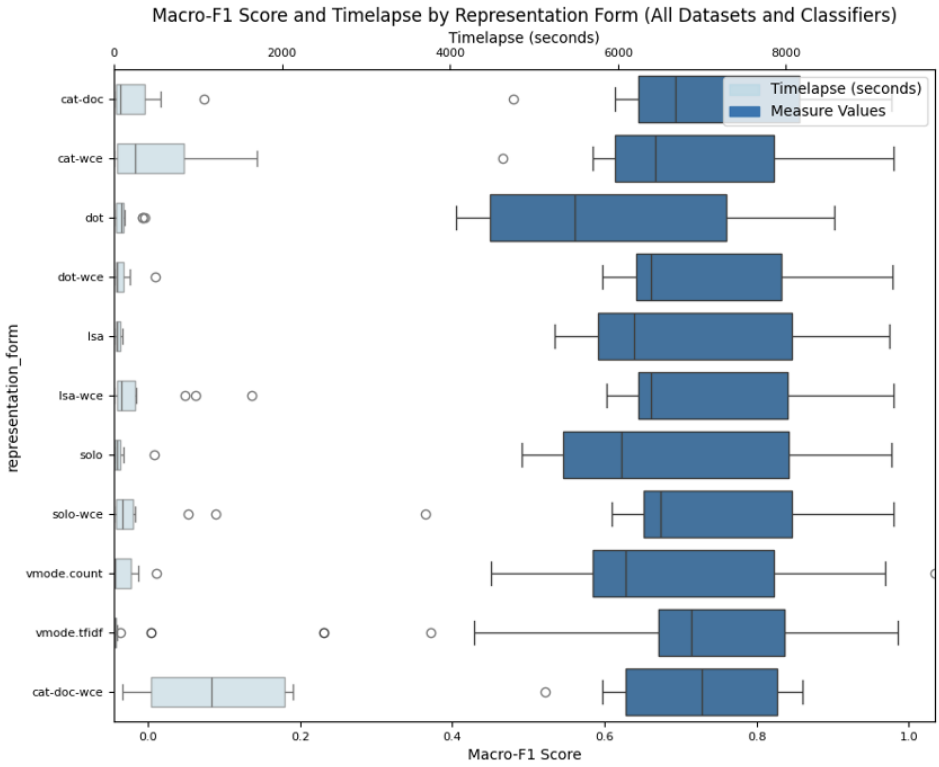
[28] The Naïve Bayes classifier, MultinomialNB from sklearn, only supports the vectorized representation form given its dependency on positive embeddings only so this is the only representation form that is computational viable.

arxiv	multi-label	58	0.611	116.04	0.614	74.89	0.221	9.24
arxiv_protoformer	single-label	10	0.803	173.36	0.811	946.06	#DIV/0!	#DIV/0!
bbc-news	single-label	5	0.969	3.24	0.954	3.10	0.968	0.09
imdb	single-label	2	0.858	132.26	0.848	84.66	0.820	5.10
ohsumed	multi-label	23	0.609	198.28	0.603	558.33	0.422	13.83
rcv1	multi-label	101	#DIV/0!	#DIV/0!	0.734	2639.73	#DIV/0!	#DIV/0!
reuters21578	multi-label	115	0.794	64.73	0.796	107.89	0.591	11.78
Summary								
Logistic Regression							0.749	107.70
SVM							0.744	557.52
Naïve Bayes							0.604	8.65

When we look at the data broken down by dataset, we can see that the training time, and the underlying values – in terms of which model is optimal - is very much dependent on the underlying data so one should experiment with different models before picking the optimal classifier, the optimal configuration clearly varies by dataset and the training times between the classifiers can vary significantly.

6.1.3 By Representation Form

When we look at the various representation forms, and evaluate their performance in the various classifiers, we find that, perhaps surprisingly, that the lsa and solo (which is the language model encoded representation of the underlying dataset), both individually and when combined (concatenated) with the WCEs, are the best performing options in terms of model efficacy. The model training times for these configurations is also quite efficient due to the fact that these representation forms are inherently a form of feature reduction as well.



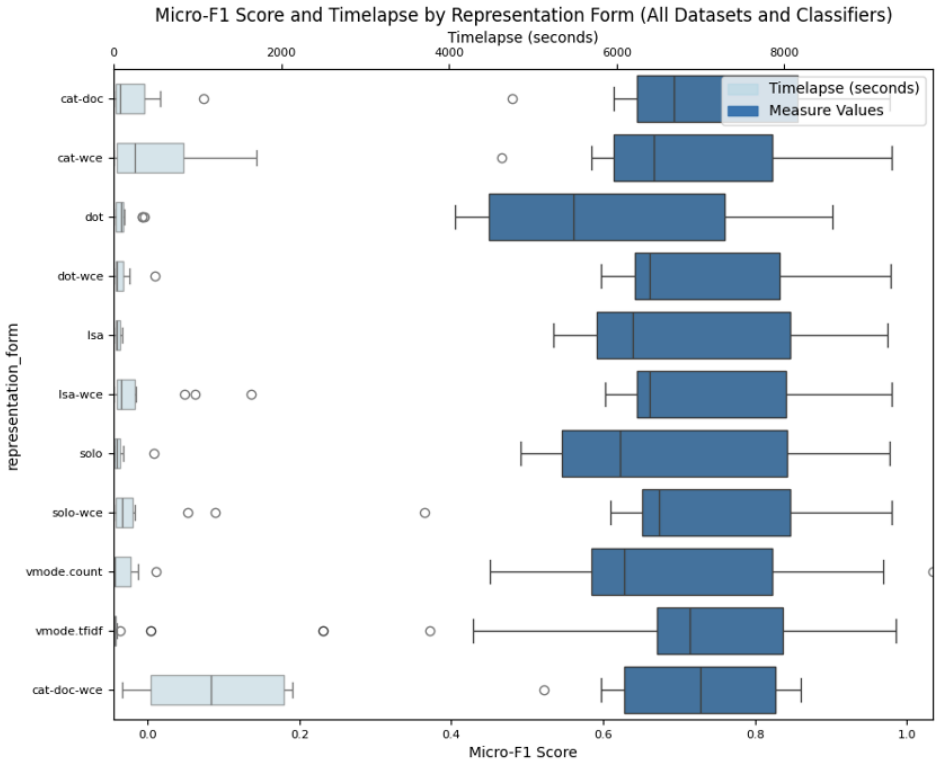


Figure 6: ML Classifier Summary: By Representation

6.1.4 By Language Model

When we look at how the individual language model types perform across the ML classifiers and datasets, we find, as illustrated in the box plot figures below, that the different language model types perform roughly the same on the various datasets, despite the fact that the word based language models are vectors of size 300 while the transformer based models are computed in a higher dimensional space – 768 for the smaller BERT, RoBERTa and DistilBERT, XLNet and GPT2 models we are using and 2048 for the Lllama model and 1536 for the DeepSeek model.

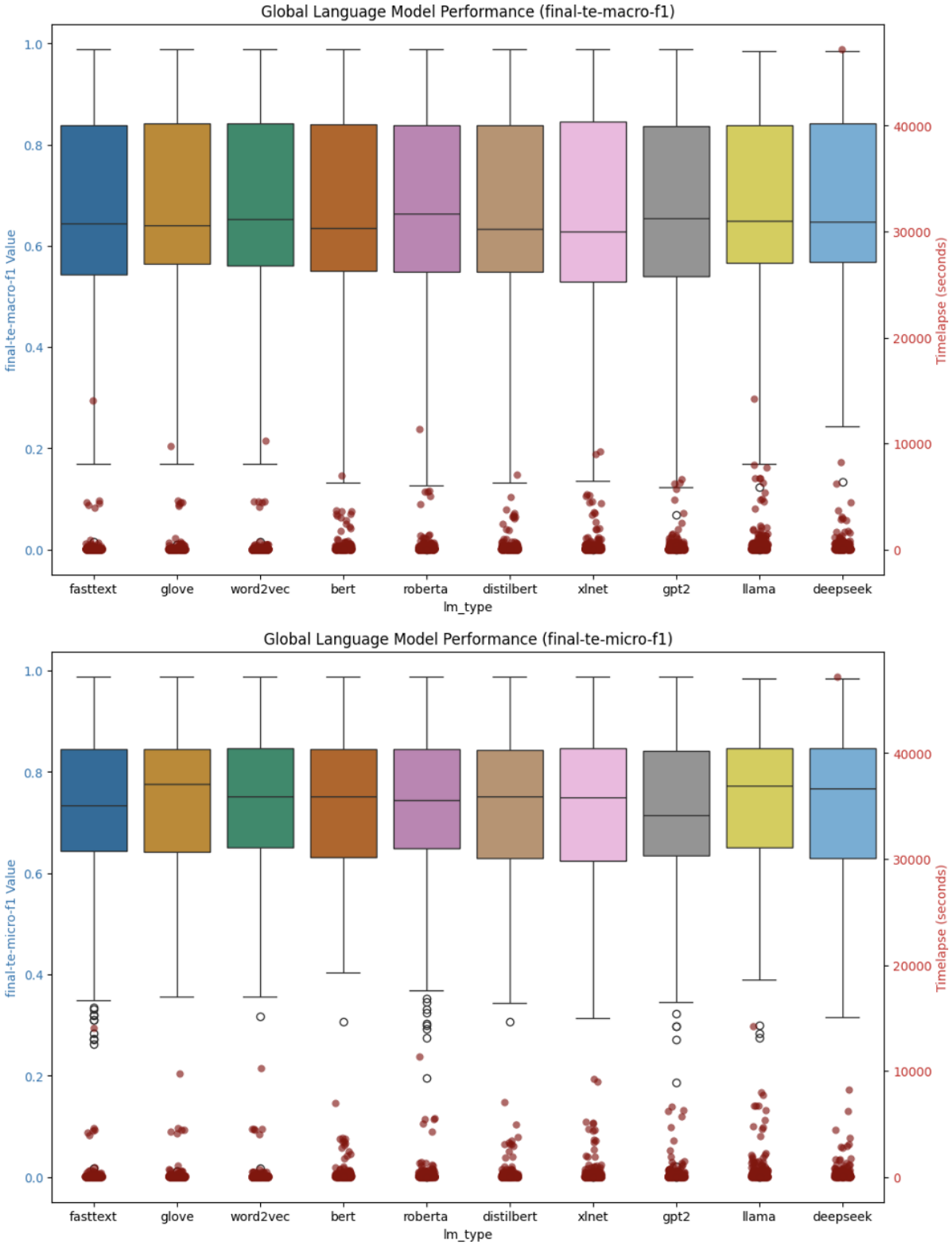


Figure 7: Language Model Performance Summary - ML Classifiers

When we look at the detail data by macro-f1 and micro-f1 statistical measures we see a similar pattern, the optimal model depends upon the dataset, but generally the measures are very similar to each other,

although the model training times are significantly more for the LR models than they are for SVM or NB classifiers.

Macro-F1 Detail

Dataset	Type	# labels	LR		SVM		NB	
			Value	Time	Value	Time	Value	Time
20newsgroups	single-label	20	0.617	72.27	0.590	56.74	0.586	11.70
arxiv	multi-label	58	0.616	116.04	0.616	74.89	0.234	9.24
arxiv_protoformer	single-label	10	0.802	173.36	0.810	946.06	#DIV/0!	#DIV/0!
bbc-news	single-label	5	0.968	3.24	0.953	3.10	0.968	0.09
imdb	single-label	2	0.858	132.26	0.846	84.66	0.821	4.90
ohsumed	multi-label	23	0.574	198.28	0.568	558.33	0.309	13.83
rcv1	multi-label	101	#DIV/0!	#DIV/0!	0.489	2704.96	#DIV/0!	#DIV/0!
reuters21578	multi-label	115	0.506	64.73	0.505	107.89	0.290	11.78

Summary	Value	Time
Logistic Regression	0.701	107.70
SVM	0.674	568.12
Naïve Bayes	0.535	8.59

Micro-F1 Detail

Dataset	Type	# labels	LR		SVM		NB	
			Value	Time	Value	Time	Value	Time
20newsgroups	single-label	20	0.625	72.27	0.596	56.74	0.610	11.70
arxiv	multi-label	58	0.611	116.04	0.614	74.89	0.221	9.24
arxiv_protoformer	single-label	10	0.803	173.36	0.811	946.06	#DIV/0!	#DIV/0!
bbc-news	single-label	5	0.969	3.24	0.954	3.10	0.968	0.09
imdb	single-label	2	0.858	132.26	0.848	84.66	0.820	5.10
ohsumed	multi-label	23	0.609	198.28	0.603	558.33	0.422	13.83
rcv1	multi-label	101	#DIV/0!	#DIV/0!	0.734	2704.96	#DIV/0!	#DIV/0!
reuters21578	multi-label	115	0.794	64.73	0.796	107.89	0.591	11.78

Summary	Value	Time
Logistic Regression	0.749	107.70
SVM	0.744	568.12
Naïve Bayes	0.604	8.65

6.2 Neural Models (Deep Learning)

Within the context of our neural model analysis, we look at performance results by classifier and by (pretrained) language models and integrate the results with model training times (timelapse field) into one graphic, focusing on the macro-f1 and micro-F1 metrics which are good, industry standard metrics to measure the performance of a classifier in both multi-label and single-label scenarios.[29]

The primary questions we are looking at answering here are:

I. *pretrained embedding performance comparison*: which pretrained language models perform the best and by how much? Do certain pretrained embeddings perform better on certain datasets? If so why? If not why not?

II. *neural architecture performance comparison*: which neural architectures perform the best in terms of model macro-f1 and micro-f1 metrics? Is this the same across all datasets or does it differ?

III. *system resource performance comparison*: how long does each model take to run and which ones are more computationally expensive? What is the best model that takes the least amount of computing resources to train?

6.2.1 By Language Model

When looking at the on average best performing language models, of the model variants that we have chosen for this work, we find that XLNet and RoBERTa stand out as top performers, with the transformer models outperforming their static counterparts, even when they were assisted with WCEs, again on average. We can see this illustrated in the language model summary macro-f1 and micro-f1 charts (box plots) below. With the box plot graphic, which looks at the left y-axis which shows the measurement value between 0 and 100 (%), with the center line representing the median value, the shaded box area the interquartile range between the 25th and 75th percentiles, and outliers represented as unshaded circles. Overlaid on these graphics is the timelapse value which shows how it took, in seconds, to build (train) the model.

We also see, again on average across all the datasets, relatively the same performance for the three static word embedding language models we test with (GloVe, Word2Vec and fastText models). Given that they are of comparable size and are trained basically on the same underlying data (Web Common Crawl), this is not a surprising result and speaks to how comparable the different training methods are to each other when used in this type of neural model / text classification setting. We see the best performance across all the datasets that we tested from the XLNet and RoBERTa language models, with BERT, GPT2 and DeepSeek coming in very close behind those two frontrunners.

[29] Specific formulas for metric calculation can be found in the Experimental Setup section of this work.

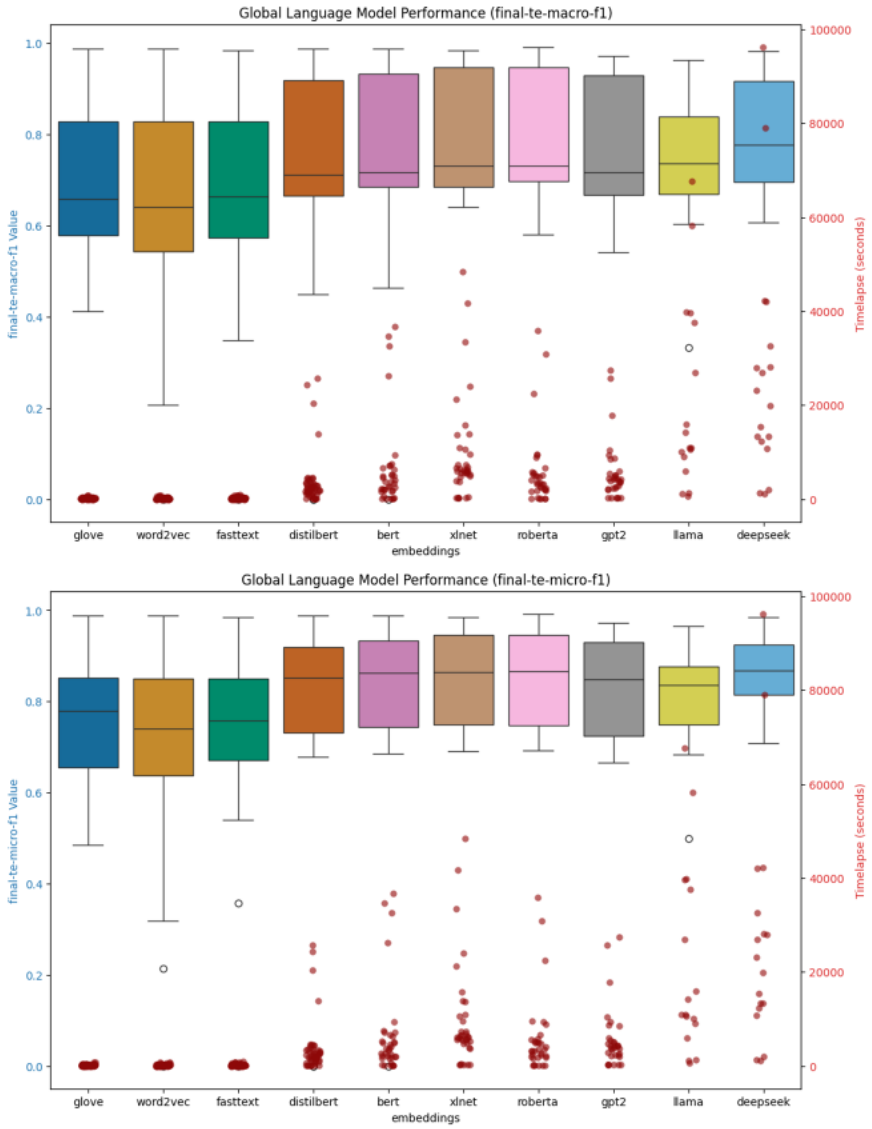


Figure 8: Neural Classifier Language Model Type Performance Summary

Notably however, from a computational resource requirements perspective (the timelapse, secondary y-axis data on the right), the static models are significantly cheaper to build. According to our analysis, the computational resource requirements of the transformer models are 50x those of their static counterparts, and this does not take into account the 4 to 1 GPU ratio that the model training of the transformers used. *This accounts for some 200x more computational resources required by the transformer model training versus the static model in our analysis.* Also, it's also important to understand that this computational costs is not just a one-time cost, in the event that you want your model to be updated on an ongoing basis, which is the case in any operational setting, that represents a 50x incremental resource cost for each and every model training update on an ongoing basis which, over time, geometrically expands the gap resource wise between what would be required to train a static embedding model versus a transformer, dynamic model.

Model Type	Avg Training Time (seconds)	Average Training Time (minutes)

Static	182.8344083	3.047240138
Transformer	9978.987402	166.3164567
	Training time Factor:	***54.5793732***

6.2.2 By Classifier

When we look at the data in aggregate by Classifier, we see the same trend from another perspective – namely that the HuggingFace transformer classifiers perform better than their static counterparts, but their training cost is significantly higher.

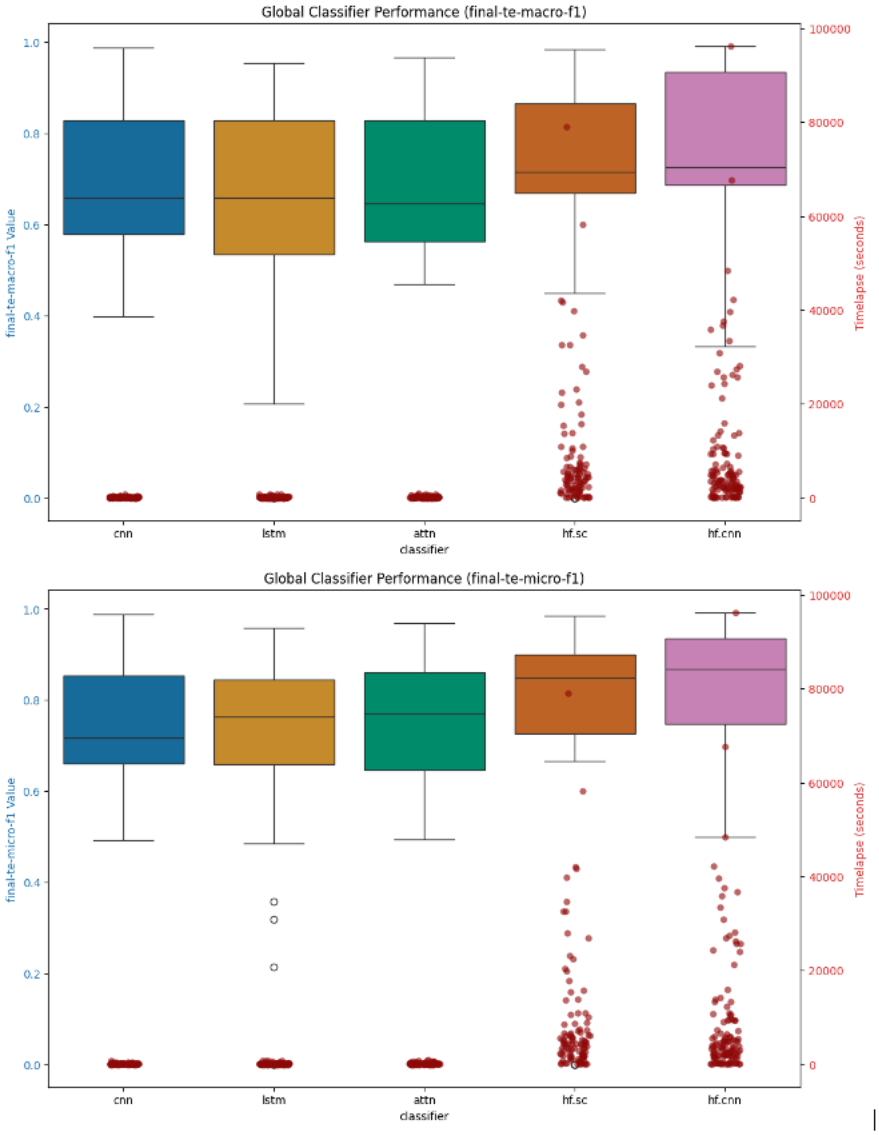


Figure 9: Neural Model Classifier Performance Summary

Between the two types of neural classifiers, on average across all the datasets, the average difference between the Macro-F1 and Micro-F1 values is .02 for macro-f1 values and .03 for micro-f1 values. The point being of course that if one is trying to select which type of neural model to use, once should consider just how significant that extra uptick in model performance is worth.

Model Type	Avg Macro-F1	Avg Micro-F1
Static	0.726	0.743
Transformer	0.746	0.772

6.2.3 By Dataset

This is of course on average, if we drill into the language model performance for a given dataset for example, we will see some variability, as illustrated in the figure below which shows the macro-f1 and micro-f1 performance for various forms of representation for the Ohsumed dataset which contains medical data and therefore is littered with many medical specific terms that would not be found in a generic language model training corpus.

We use the *representation* shorthand to describe the classifier hyperparameter settings and language representation form (left y-axis) which we append the embedding layer shape (dimensions) at the end so we get a sense as to the size of the embedding dictionary we are using, at least at model initialization time. We also have timelapse data on the same chart (right y-axis) with the actual the measure value on the bottom of the chart on the x axis.



Figure 10: Macro-F1 scores for neural classifiers (Ohsumed dataset)

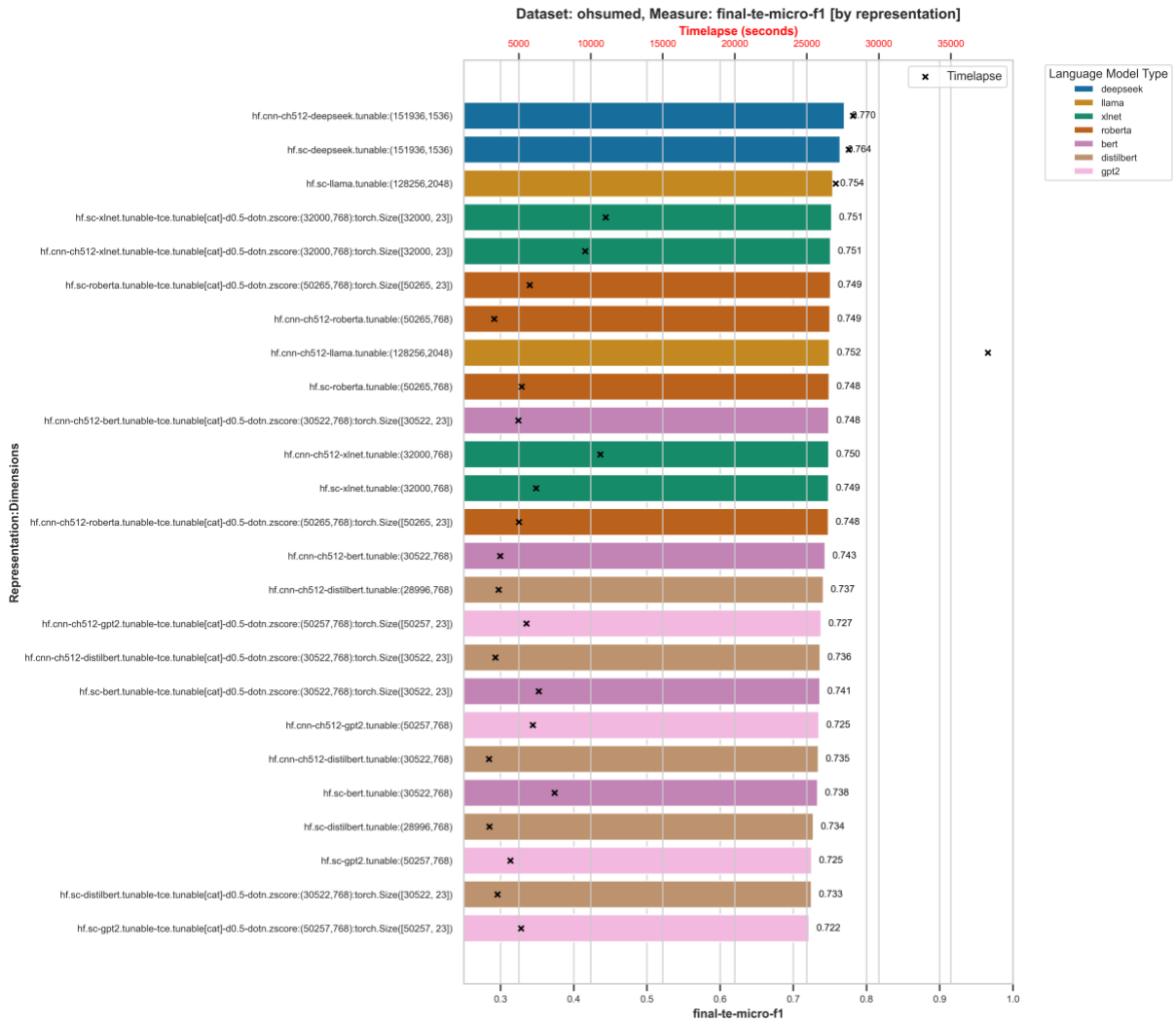


Figure 11: Micro-F1 scores for neural classifiers (Ohsumed dataset)

Of the top 25 models, we can see that none of the static models even made the list, with the DeepSeek and Llama models slightly edging out the rest of the transformer models for both macro-f1 and micro-f1 statistics.

When we look at the RCV1 dataset macro-f1 and micro-f1 combined charts, news data which the language models were no doubt pre-trained on (from a categorical perspective), we see again the transformer models dominating from a performance standpoint, with the HF CNN classifier performing the best, on average.

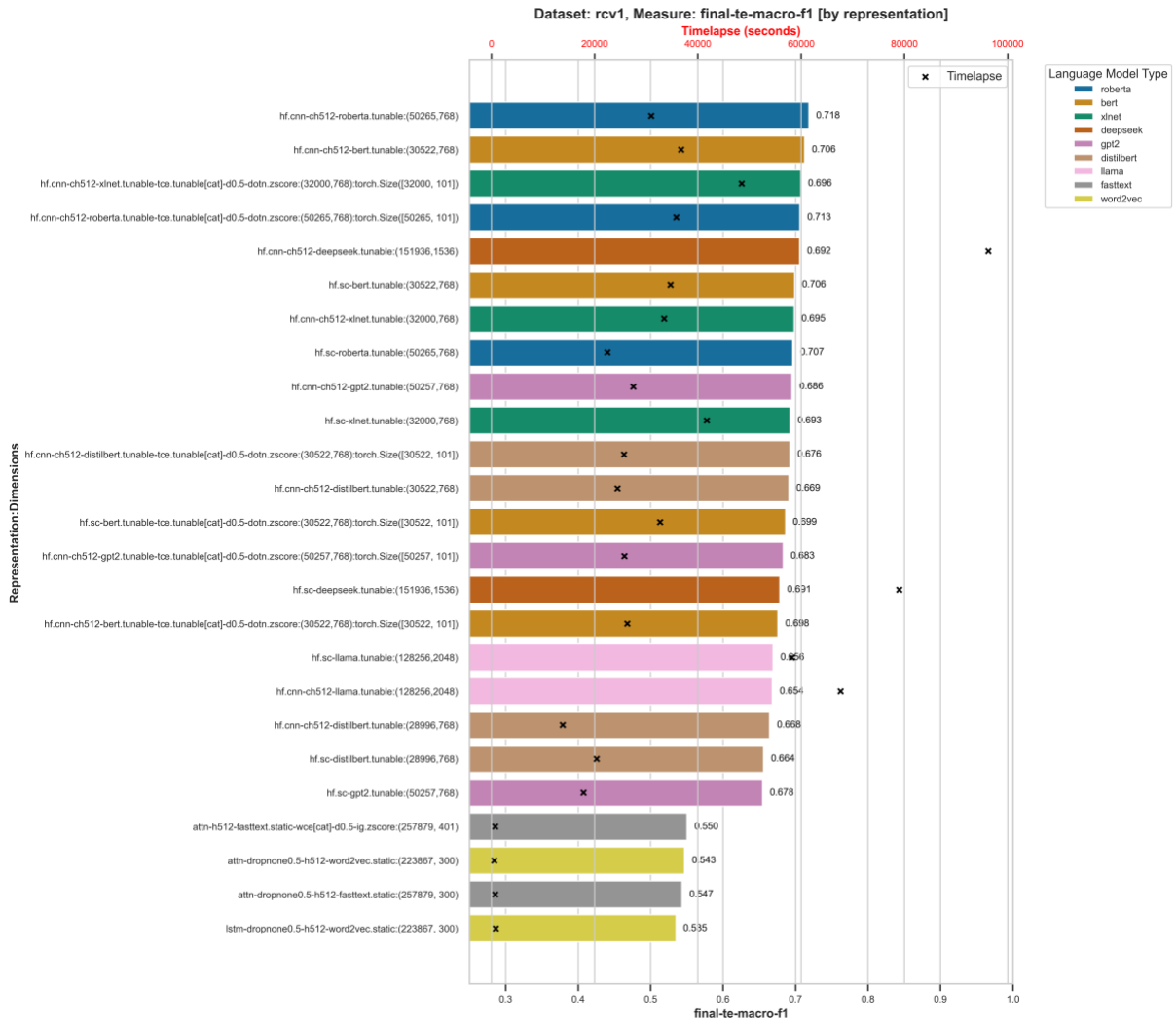


Figure 12: Macro-F1 values across neural classifiers (RCV1 dataset)

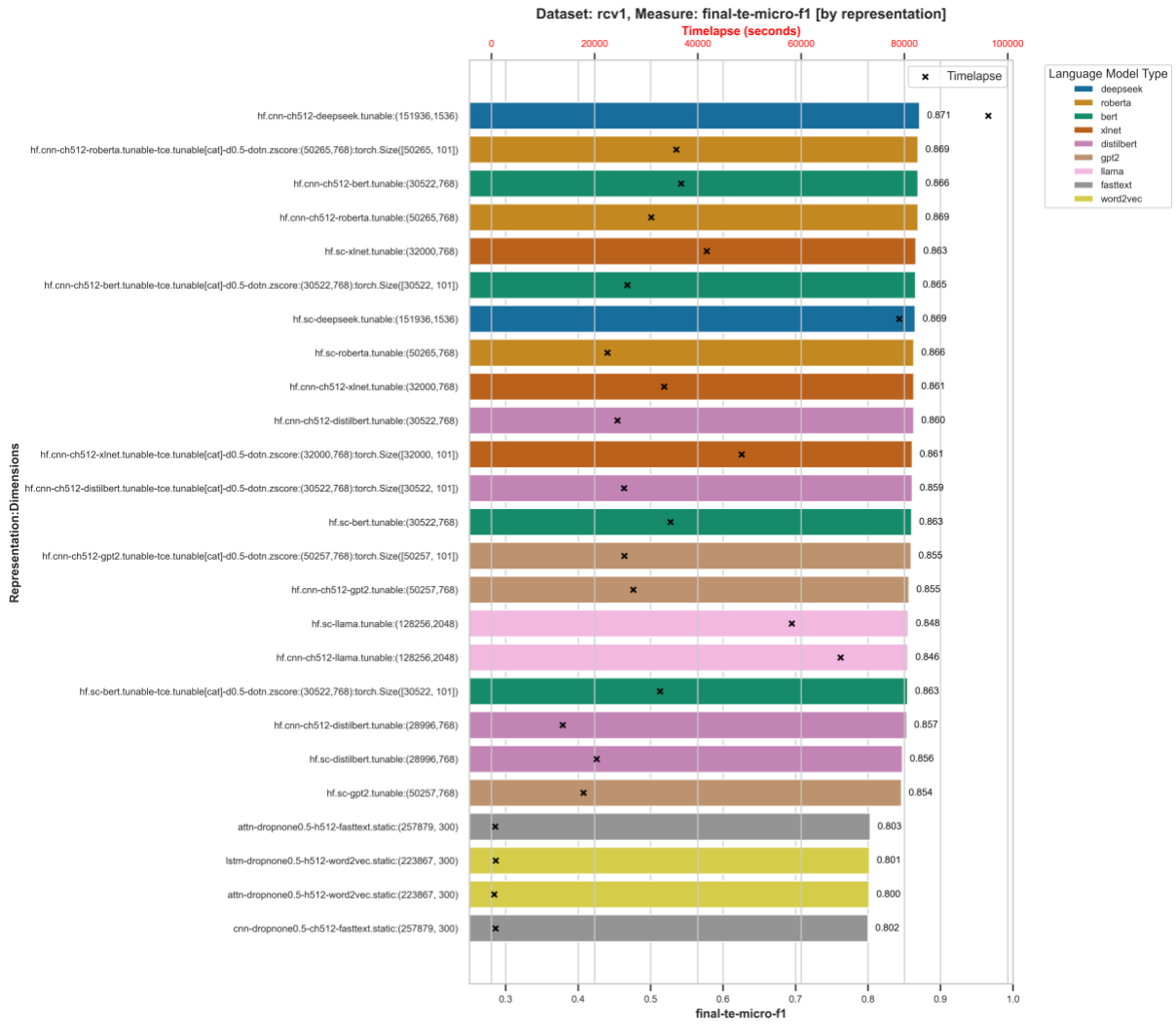


Figure 13: Micro-F1 values across neural classifiers (RCV1 dataset)

For the rcv1 dataset, we can see a significant gap between the performance of the best transformer models, .718 for *hd.cnn-roberta:(50265,768)* for macro-f1 and .871 for *hf.cnn-deepseek:(151936, 1536)* for micro-f1, while the static pretrained embedding classifiers did not get past .56 for macro-f1 and .81 for micro-f1 respectively.

When we look at the arxiv data, which is academic research papers (which should be included in most transformer model training datasets but not necessarily in the static pretrained embedding models), we see a similar trend, with DeepSeek again performing better than the rest of the transformer models across both macro-f1 and micro-f1 measures and the static embedding neural models, even with the Word-Class Embeddings, lagging behind.

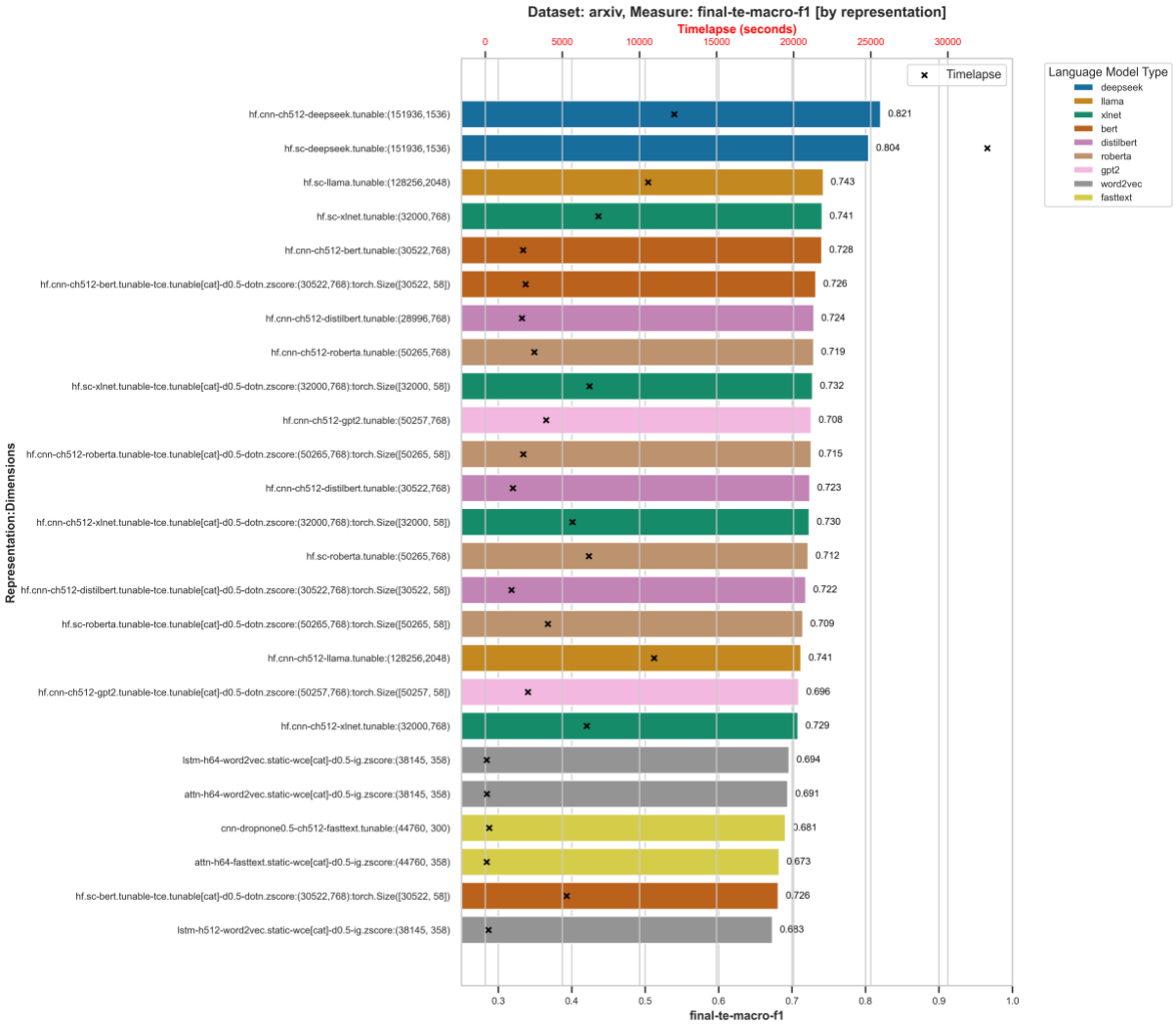


Figure 14: arxix neural classifier performance comparison (Macro-F1)

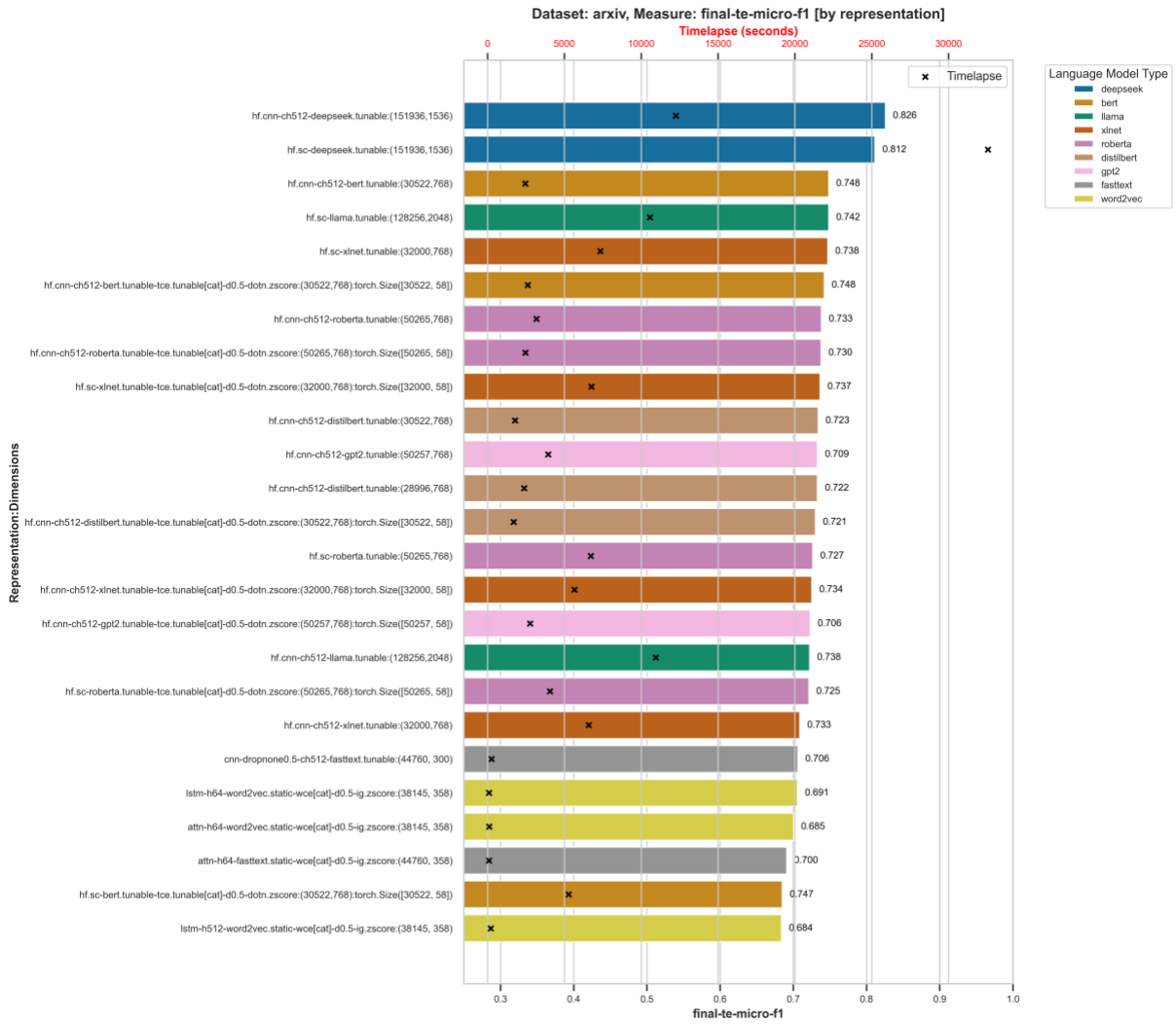


Figure 15: arxix neural classifier performance comparison (Micro-F1)

Over a 10% gap between the best static embedding model and the best performing transformer model in fact, for both the macro-f1 and micro-f1 measures, although the best performing static embedding language model classifiers came close to some of the worst performing transformer models, within .30%.

So even though the incremental improvement on average between the static versus the dynamic (transformer) models is .02% for the macro-f1 measure across all models and %.03 for micro-f1, when we drill into the individual classifier performance statistics at the dataset level, we see a significant gap between the best performing transformer classifiers (the HuggingFace Sequence Classifier typically) and the best performing static pretrained embedding classifier, even with the Word-Class Embeddings for improved performance.

6.3 Global Analysis

We now look at the results across the entire range of Layer Cake tests – the machine learning classifiers, the static neural embedding classifiers, and the transformers to see what patterns emerge from the data. When

we look at the entire picture, the performance of all of the classifiers against each other, we see what we might have expected, that the transformer models perform – again on average – better than their static model counterparts, as well as their ML classifier counterparts. What is perhaps somewhat surprising is that the ML classifiers perform just as well as the static neural models.

The training times across the models again lines up with what we would expect, with the transformer models averaging somewhere in the neighborhood of 20,000 seconds to train, which is almost 6 hours, with more than a few outliers which took greater than 10 hours to train. Almost all of the ML and static embedding neural models were under 6 hours and most of them were under an hour.

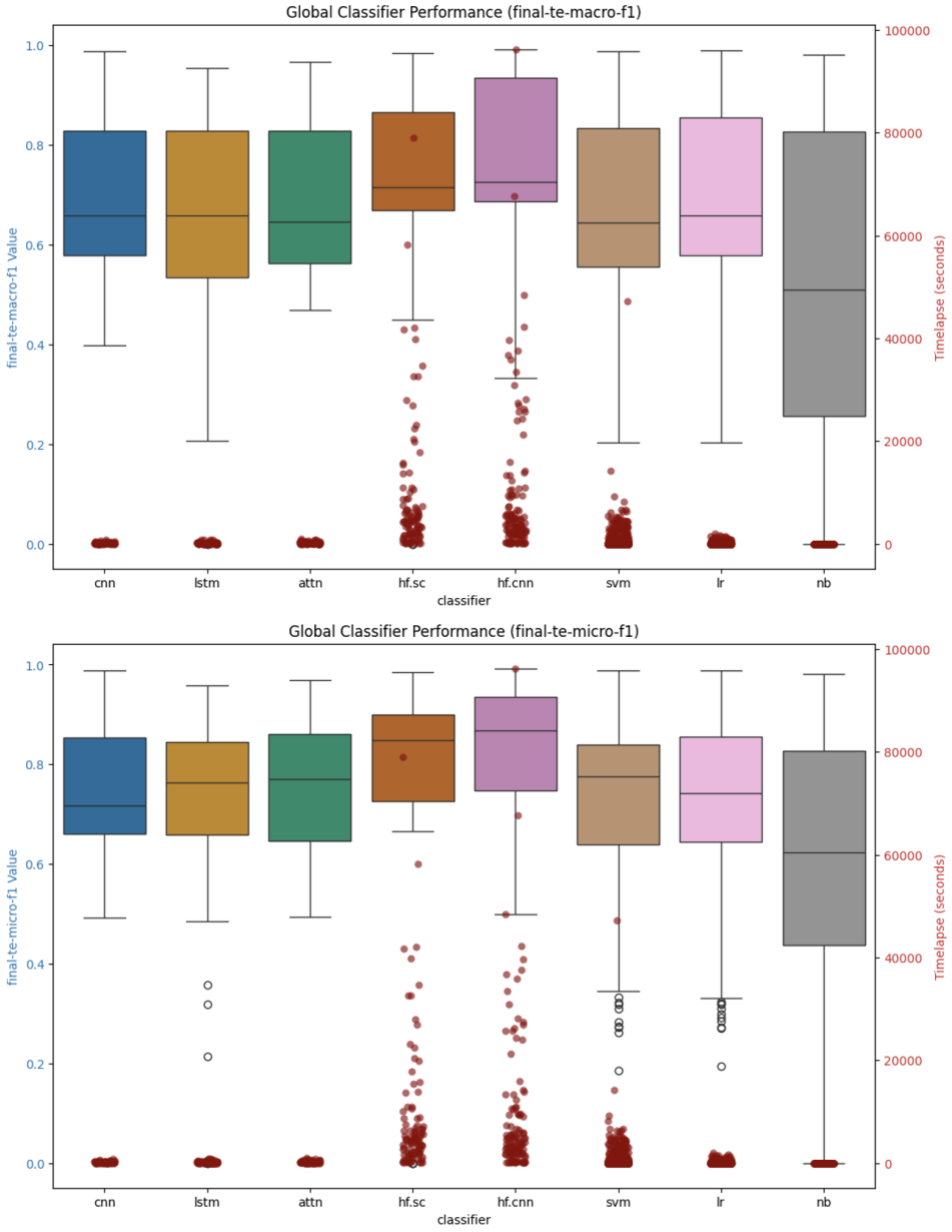


Figure 16: Global Classifier Performance Summary

When we look at the global view by language model, we find perhaps something is surprising. All of the language models, when looked at through the lens of all the datasets and classifiers, perform almost exactly the same, while the larger models – as you might expect – take longer to train.

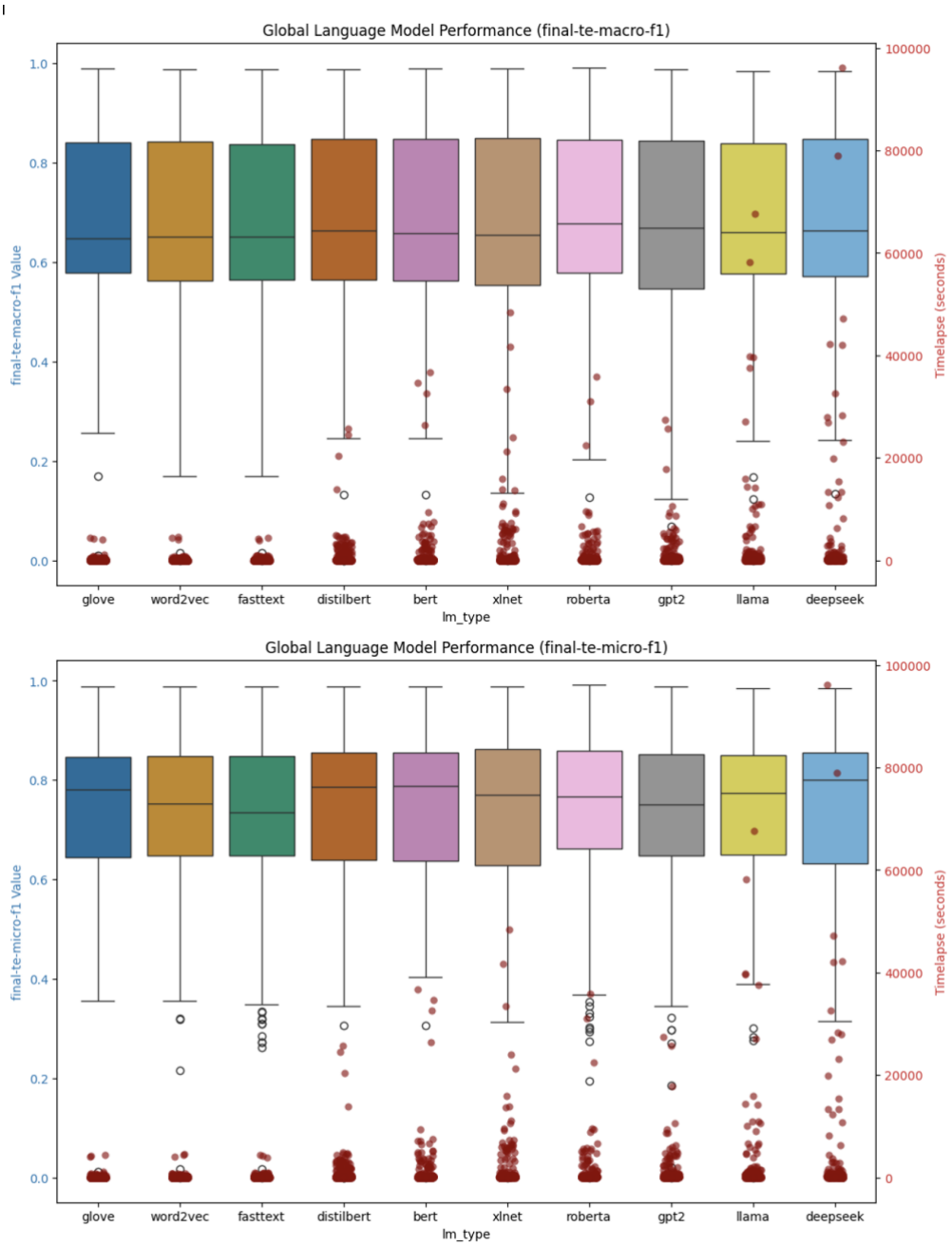


Figure 17: Global Performance Summary by Language Model

If we look at individual datasets however and look at how the different classifiers perform relative to each other (given that it's fair to say that it is the classifier architecture rather than the underling language model that drives the bulk of the performance of the classifiers relatively speaking), we see a slightly more nuanced and perhaps surprising result emerge.

For example if we look at the top 10 results for macro-f1 and micro-f1 measures for a few datasets – specifically we look at the result detail associated with the 20newsgroups and reuters21578 datasets which represented the base single-label and multi-label test datasets in our research, and then in addition we look at the arxiv_protoformer and Ohsumed datasets, single-label and multi-label examples respectively that both contain a lot of technically specific language that has a high(er) probability of not being included in training sets for the various language models we are testing with, in particular the static embedding models which are trained on common crawl data primarily.

dataset	classifier	lm_type	representation	value	timelapse
20newsgroups	hf.sc	deepseek	solo	0.706	15468.50
20newsgroups	hf.cnn	bert	solo	0.704	1875.76
20newsgroups	hf.cnn	distilbert	solo	0.702	2323.23
20newsgroups	*lr*	llama	solo-wce	0.701	36.34
20newsgroups	hf.sc	bert	solo	0.699	2659.65
20newsgroups	hf.cnn	roberta	solo	0.698	1834.15
20newsgroups	*cnn*	fasttext	solo	0.694	110.54
20newsgroups	hf.cnn	deepseek	solo	0.693	10835.79
20newsgroups	hf.sc	xlnet	solo	0.689	5236.63
20newsgroups	hf.sc	roberta	solo	0.689	1690.47
reuters21578	hf.sc	xlnet	solo	0.665	6270.53
reuters21578	hf.cnn	xlnet	solo	0.659	6531.26
reuters21578	hf.sc	deepseek	solo	0.651	23166.31
reuters21578	hf.cnn	deepseek	solo	0.647	13317.33
reuters21578	hf.cnn	roberta	solo	0.619	4912.37
reuters21578	hf.cnn	bert	solo	0.606	5075.94
reuters21578	hf.cnn	llama	solo	0.606	15962.77
reuters21578	*svm*	glove	cat-doc	0.606	19.01
reuters21578	*svm*	fasttext	cat-doc	0.606	16.72
reuters21578	hf.cnn	gpt2	solo	0.606	4736.67
arxiv_protoformer	hf.sc	deepseek	solo	0.865	42047.7
arxiv_protoformer	hf.cnn	deepseek	solo	0.864	42243.9
arxiv_protoformer	*lr*	llama	solo	0.856	184.9
arxiv_protoformer	*lr*	deepseek	solo	0.855	154.6
arxiv_protoformer	hf.cnn	xlnet	solo	0.852	21222.4
arxiv_protoformer	hf.cnn	distilbert	solo	0.851	4579.3
arxiv_protoformer	*svm*	deepseek	solo	0.850	238.7
arxiv_protoformer	*svm*	deepseek	cat-doc	0.850	1571.7
arxiv_protoformer	hf.sc	bert	solo	0.850	6661.2
arxiv_protoformer	hf.sc	distilbert	solo	0.849	4506.9

ohsumed	hf.cnn	deepseek	solo	0.748	28201.4
ohsumed	hf.sc	deepseek	solo	0.745	27905.8
ohsumed	hf.sc	llama	solo	0.740	27007.4
ohsumed	hf.cnn	llama	solo	0.736	37576.8
ohsumed	hf.cnn	xlnet	solo	0.735	9609.6
ohsumed	hf.sc	roberta	solo	0.734	5755.1
ohsumed	hf.sc	xlnet	solo	0.734	11027.5
ohsumed	hf.cnn	roberta	solo	0.733	5001.3
ohsumed	hf.cnn	bert	solo	0.733	4968.2
ohsumed	hf.cnn	gpt2	solo	0.719	5970.5

dataset	classifier	lm_type	representation	value	timelapse
20newsgroups	hf.cnn	bert	solo	0.713	1875.76
20newsgroups	lr	llama	solo-wce	0.713	36.34
20newsgroups	hf.cnn	distilbert	solo	0.711	2323.23
20newsgroups	hf.cnn	deepseek	solo	0.710	10835.79
20newsgroups	hf.cnn	roberta	solo	0.709	1834.15
20newsgroups	hf.sc	deepseek	solo	0.708	15468.50
20newsgroups	hf.sc	bert	solo	0.705	2659.65
20newsgroups	cnn	fasttext	solo	0.705	110.54
20newsgroups	hf.sc	xlnet	solo	0.702	5236.63
20newsgroups	svm	llama	solo-wce	0.702	133.89
reuters21578	attn	fasttext	solo	0.867	240.57
reuters21578	attn	glove	solo	0.860	204.13
reuters21578	attn	word2vec	solo	0.862	231.05
reuters21578	cnn	fasttext	solo	0.856	59.22
reuters21578	cnn	glove	solo	0.845	122.65
reuters21578	cnn	word2vec	solo	0.851	109.41
reuters21578	hf.cnn	bert	solo	0.897	5075.94
reuters21578	hf.cnn	deepseek	solo	0.896	13400.41
reuters21578	hf.cnn	distilbert	solo	0.890	1986.27
reuters21578	hf.cnn	gpt2	solo	0.887	4736.67
arxiv_protoformer	hf.sc	deepseek	solo	0.865	42047.65
arxiv_protoformer	hf.cnn	deepseek	solo	0.865	42243.94
arxiv_protoformer	lr	llama	solo	0.856	184.94
arxiv_protoformer	lr	deepseek	solo	0.855	154.60
arxiv_protoformer	hf.cnn	xlnet	solo	0.852	21222.40

arxiv_protoformer	hf.cnn	distilbert	solo	0.852	4579.25
arxiv_protoformer	hf.sc	bert	solo	0.851	6661.19
arxiv_protoformer	svm	deepseek	solo	0.850	238.68
arxiv_protoformer	svm	deepseek	cat-doc	0.850	1571.72
arxiv_protoformer	svm	llama	solo	0.849	315.12
ohsumed	hf.cnn	deepseek	solo	0.770	28201.37
ohsumed	hf.sc	deepseek	solo	0.764	27905.77
ohsumed	hf.sc	llama	solo	0.754	27007.43
ohsumed	hf.sc	xlnet	solo	0.752	11027.47
ohsumed	hf.cnn	xlnet	solo	0.751	9609.57
ohsumed	hf.sc	roberta	solo	0.751	5755.08
ohsumed	hf.cnn	roberta	solo	0.750	3289.45
ohsumed	hf.cnn	llama	solo	0.749	37576.85
ohsumed	hf.cnn	bert	solo	0.749	4968.19
ohsumed	hf.cnn	distilbert	solo	0.741	3592.11

In all but the Ohsumed dataset case in fact, there exists a top performing ML classifier that builds very quickly (low timelapse value for training) that has a competitive value for both macro-f1 and micro-f1 relative to the neural models we use for static embeddings as well as the transformer models. While this result is consistent with other results along the same lines (Wahba, et al., 2023), the result is nonetheless surprising and reinforced given the breadth and depth of the testing involved with this research.

7 Summary & Concluding Remarks

Classifying natural language into predefined categories is a classical problem in NLP that has applicability across a wide range of industries, the foundations of which are instrumental in solving problems around topic modeling and sentiment analysis in news and online media, threat and anomaly detection in cyber security, disease diagnosis in biology, genomics and healthcare. It is a foundational NLP problem, straightforward from an AI perspective but nonetheless serves as a good test framework for studying the relative performance of different types of language models, as well as classifiers - the main thrust of this study.

With this work we create a platform for the testing of text classification models across a wide variety of datasets and language models (Layer Cake), focusing on model performance metrics in terms of efficacy (how well the data is classified), as well as performance (what are the computational requirements for model construction), and in so doing we have shed light on the opportunity cost of working with LLMs versus more simple models and by doing so help both researchers as well as practitioners to hone in on the right model for the right problem – one of the most important facets of building out any ML or AI based solution.

With the introduction of Layer Cake and this research, we look to focus the attention specifically on this particular question – namely how much better do these LLMs actually perform on a foundational NLP problem like text classification, relative to their more classical, and less computationally intensive, predecessors, and as such reframe the conversation around how and when LLMs should be used to solve practical, real-world, NLP solves for businesses where cost, and time, are important ingredients for successful outcomes. In short, in this work we look to answer the question, again within the context of text classification specifically but with broader implications on how these LLMs are being used to solve NLP problems more generally, *what kind of performance improvement can we expect to gain by using these LLMs and at what (computational) cost?*

What we find is that in fact, consistent with research on LLM performance in NLP, the best performing models are in fact LLMs, but a) they cost significantly more time and money to train and b) in almost all cases there exists a classical ML, CPU bound alternative that performs almost as well as best transformer models for that particular dataset. The exception to this is the Ohsumed dataset, which is highly specific to the healthcare and medical industry and as such includes a lot of very specific, technical terms. In this scenario, and this conclusion can undoubtedly be broadened around classes of NLP problems that work with technically specific language, i.e. termed *out of vocabulary* terms in the NLP literature, it seems evident that the LLMs handle this language text better than their static embedding language model counterparts (like Word2Vec, GloVe or fastText) and better than their ML classifier predecessors which do not have a good way to build dynamic embedding structures which can account for this type technically specific jargon which is not part of the common core language set in daily use which is primarily what these pretrained language models are trained on.

What we also find, is that Word-Class Embeddings, which have been shown to improve the performance of text classification models in both ML classifier context as well as static embedding neural models, are not at all effective in a transformer, context-aware embedding environment. This is in all likelihood due to a combination of a) the already granular, context-aware nature of the transformer models which include this type of word/token to class type information, and b) the transformer-based models are designed to be self-contained and are not set up to be augmented with additional embedding information that comes from outside the model.

When we look at the various forms of language representation within the context of the ML testing section of this work, we do find two very effective feature reduction techniques that stand out in terms of performance. One is the tried and true Latent Semantic Indexing approach which uses matrix factorization techniques to identify the most salient features (words or tokens) in a given dataset representation, and another – perhaps more surprisingly – is the approach that uses the embedding representation form of the underlying dataset, either with or without the use of Word (or Token as the case may be) Class Embeddings. WCEs / TCEs were also shown to be effective when combined with the output of an LSA reduction of the vectorized text as well. Both of these approaches to language representation stood up very well, and in most cases outperformed, the standard tf-idf vector representation forms, both in terms of efficacy (macro and micro f1 scores), as well as in terms of performance and computational complexity.

7.1 Areas of Promising Research

While the testing we've done with Layer Cake covers a broad spectrum of classifiers and language model forms, we nonetheless limit the study to Euclidean spatial geometry, i.e. all of the language representation forms we work with are constructed in Euclidean spaces. However, there is a lot of promising research that has been done with hyperbolic geometry (Tifrea, et al., 2019) (Leimeister & Wilson, 2019) (Valentino, et al., 2024) and it would be very interesting to see how these embedding forms performed relatively speaking, or to see if using these embeddings that are trained in hyperbolic spaces, which lend themselves to a more hierarchical structure, could be used in conjunction with some of the other language representation forms we test with (like we do with WCEs) and what kind of effect performance wise that had on the models.

Part of what we look to shed light on in this work is the different computational requirements of the different models, in particular in light of the fact that the underlying hardware dependencies for the different models are different – with the machine learning classifiers being CPU bound and the neural models being GPU bound. This of course means, and this is especially true given the underlying cost of a high-end GPU versus a high-end CPU, that we are not dealing with apples to apples comparisons when we look at the performance of the classical ML (CPU bound) vs neural (GPU bound) classifiers with respect to training time (the timelapse value in Layer Cake). GPUs are, at present at least, particularly the high-end GPUs, more costly than high end CPUs and require specific software libraries to be used effectively – like for example CUDA which is used throughout this work to test the neural classifiers. When building neural classifiers, the only choice to build the models at scale is to use a parallel GPU environment, aka a High Performance Computing environment, and then use the underlying software libraries – PyTorch in our case – to manage the parallelism across the GPUs. It's worth noting that given the size of LLMs, in almost all cases except the smallest of the models, a parallel GPU computational environment is necessary to do any computation at all as the models themselves will not fit on a single GPU.

From a performance standpoint, it would also be interesting to see what the effect might be of using different parallelism approaches to transformer model training to see what effect that had on model training times and ultimately GPU computational requirements which are a significant part of the training or fine tuning of LLMs. In our case, we used the default Data Parallel (DP) approach which comes built in to the Hugging Face APIs but the *Distributed Data Parallel* (DDP) PyTorch capabilities are supposed to surpass those of the more straightforward DP form of parallelism so it would be interesting to see what this difference was performance wise in terms of model training. Using DPP would also allow the testing of larger LLMs, LLMs that could be spread across multiple GPUs – models that could not be included in this study due to single GPU memory constraints that prevent the use of larger models in a DP type of setup.

From a language representation perspective, in particular with respect to the Deep Learning approaches we cover with Layer Cake, it would be interesting to see if we could combine vectorization based approaches with the standard token indexing approaches to the model, particularly within the context of classification problems as that would give us an opportunity perhaps – like with tf-idf vectorization – to weight the different tokens alongside the attention based masking that the transformer models are known for. This has been shown to be an effective approach for certain NLP problems, like sentiment analysis, as exemplified in (Liang & Niu, 2022) (Zhao, 2022) (Kamyab, et al., 2021).

Each of these areas will no doubt prove to be fruitful areas of research in the coming years as both the research community and practitioner community of language models looks to both improve and optimize the performance of LLMs across the entire spectrum of NLP, which includes point solutions which involve custom models and language representation approaches as well as field of language generation, a field whose promise continues to drive innovation and investment in various forms of language representation.

8 Bibliography

Bai, J., 2023. Qwen Technical Report. *arxiv,* p. https://doi.org/10.48550/arXiv.2309.16609.

Bojanowski, P., Grave, E., Joulin, A. & Mikolov, T., 2017. Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics,* 01 06, Volume 5, pp. 135-146; https://doi.org/10.1162/tacl_a_00051.

Brown, T. B. et al., 2020. *Language Models are Few-Shot Learners.* Red Hook, NY, USA, Curran Associates Inc., pp. 1877-1901.

Dai, Z. et al., 2019. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. *arxiv,* 2 June.p. https://doi.org/10.48550/arXiv.1901.02860.

Kamyab, M., Liu, G. & Adjeisah, M., 2021. Attention-Based CNN and Bi-LSTM Model Based on TF-IDF and GloVe Word Embedding for Sentiment Analysis. *Applied Sciences,* 11(23), p. https://doi.org/10.3390/app112311255.

Leimeister, M. & Wilson, B. J., 2019. Skip-gram word embeddings in hyperbolic space. *arxiv,* 27 May.

Liang, M. & Niu, T., 2022. Research on Text Classification Techniques Based on Improved TF-IDF Algorithm and LSTM Inputs. *Procedia Computer Science,* Volume 208, pp. 460-470; https://doi.org/10.1016/j.procs.2022.10.064.

Li, Q. et al., 2022. A Survey on Text Classification: From Traditional to Deep Learning. *ACM Transactions on Intelligent Systems and Technology (TIST),* 8 April, 13(2), pp. 1-41; https://doi.org/10.1145/3495162.

Mikolov, T., Chen, K., Corrado, G. & Dean, J., 2013. *Efficient Estimation of Word Representations in Vector Space.* s.l., arXiv, p. https://doi.org/10.48550/arXiv.1301.3781.

Minaee, S. et al., 2021. Deep Learning--based Text Classification: A Comprehensive Review. *ACM Computing Surveys (CSUR),* April, 54(3), pp. 1-40; https://doi.org/10.1145/3439726.

Moreo, A., Esuli, A. & Sebastiani, F., 2021. Word-class embeddings for multiclass text classification. *Data Mining and Knowledge Discovery,* May, Volume 35, pp. 911-963. https://doi.org/10.1007/s10618-020-00735-3.

Pennington, J., Socher, R. & Manning, C. D., 2014. *GloVe: Global Vectors for Word Representation.* Doha, Qatar, Association for Computational Linguistics, pp. 1532-1543. 10.3115/v1/D14-1162.

Qiu, X. et al., 2020. Pre-trained models for natural language processing: A survey. *Science China Technological Sciences,* 15 September, 63(10), pp. 1872-1897; https://doi.org/10.1007/s11431-020-1647-3.

Tifrea, A., Bécigneul, G. & Ganea, O.-E., 2019. *Poincaré GloVe: Hyperbolic Word Embeddings.* New Orleans, Louisiana, s.n.

Touvron, H. et al., 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv,* p. https://doi.org/10.48550/arXiv.2302.13971.

Turing, A., 1950. Computing Machinery and Intelligence. *Mind,* 59(236), pp. 433-460; 10.1093/mind/lix.236.433.

Valentino, M., Carvalho, D. S. & Freitas, A., 2024. *Multi-Relational Hyperbolic Word Embeddings from Natural Language Definitions.* St Julian's, Malta, Association for Computational Linguistics, pp. 23-34; https://doi.org/10.48550/arXiv.2305.07303.

Wahba, Y., Madhavji, N. & Steinbacher, J., 2023. *A Comparison of SVM Against Pre-trained Language Models (PLMs) for Text Classification Tasks*. Cham, Switzerland, Springer Nature, pp. 304-313; https://doi.org/10.1007/9.

Worth, P., 2023. Word Embeddings and Semantic Spaces in Natural Language Processing. *International Journal of Intelligence Science,* 13(1), pp. 1-21. https://doi.org/10.4236/ijis.2023.131001.

Yang, Z. et al., 2019. *XLNet: generalized autoregressive pretraining for language understanding*. Red Hook, NY, Curran Associates Inc., p. 5753–5763.

Zhao, H., 2022. Research of Text Classification Based on TF-IDF and CNN-LSTM. *Journal of Physics: Conference Series,* January, 2171(1), pp. 10.1088/1742-6596/2171/1/012021.